

Java - Final - Adhish.docx

 Islington College, Nepal

Document Details

Submission ID

trn:oid:::3618:110039437

Submission Date

Aug 29, 2025, 10:27 AM GMT+5:45

Download Date

Aug 29, 2025, 10:29 AM GMT+5:45

File Name

Java - Final - Adhish.docx

File Size

48.3 KB

16 Pages

6,144 Words

38,560 Characters



Page 2 of 19 - Integrity Overview

Submission ID trn:oid:::3618:110039437

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **14 Not Cited or Quoted** 2%
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations** 0%
Matches that are still very similar to source material
-  **0 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 2%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Table of Contents

1. Introduction	7
2. Aims and Objectives	7
3. GUI Wireframes	8
4. Class Diagram	12
4.1. ArtGalleryDiagram Class Diagram	12
4.2. StandardVisitor Class Diagram	13
4.3. EliteVisitor Class Diagram	13
4.4. Final Class Diagram	14
5. Pseudocode	15
5.1. NavBar Button	15
5.2. Ticket Type Combo - Login	15
5.3. Add Visitor - Login	15
5.4. Clear - Login	18
5.5. Log Visit - Login	19
5.6. Clear – Visitors	21
5.7. Assign Advisor (ELITE) – Visitors	21
5.8. Check Upgrade (STANDARD) – Visitors	23
5.9. Read File – Visitors	24
5.10. Buy Artwork – Artworks	27
5.11. Cancel Artwork – Artworks	28
5.12. Generate Bill – Artworks	30
5.13. Clear – Artworks	32
5.14. Calculate Discount – Artworks	32
5.15. Calculate Reward – Artworks	34
6. Method Descriptions	36
6.1. ArtGalleryVisitor Methods	36
6.2. StandardVisitor Methods	50
6.3. EliteVisitor Methods	56
7. Testing	62

7.1. Test 1a: Verify Program Compilation.....	62
7.2. Test 1b: Verify GUI Launch.....	63
7.3. Test 2a: Add Standard Visitor.....	65
7.4. Test 2b: Add Elite Visitor.....	67
7.5. Test 3a: Log Visit.....	69
7.6. Test 3b: Buy Product.....	71
7.7. Test 3c: Cancel Product.....	72
7.8. Test 3d: Assign Personal Advisor (Elite).....	73
7.9. Test 4a: Check Discount Upgrade.....	75
7.10. Test 4b: Calculate Discount.....	77
7.11. Test 4c: Calculate Reward Points.....	79
7.12. Test 5a: Save to File.....	80
7.13. Test 5b: Read from File.....	81
8. Error Detection and Correction.....	82
8.1. Syntax Error.....	82
8.2. Runtime NPE in “Read File” Dialog.....	85
8.3. Logical Refund Calculation (Standard Cancel).....	86
9. Conclusion.....	88
10. References.....	88
11. Appendix.....	89

Table of Figures

Figure 1: Wireframe Dashboard	8
Figure 2: GUI Dashboard	8
Figure 3: Wireframe Login	9
Figure 4: GUI Login	9
Figure 5: Wireframe Visitors	10
Figure 6: GUI Visitors	10
Figure 7: Wireframe Artworks	11
Figure 8: GUI Artworks	11
Figure 9: ArtGalleryVisitor Class Diagram	12
Figure 10: StandardVisitor Class Diagram	13
Figure 11: EliteVisitor Class Diagram	13
Figure 12: Final Class Diagram	14
Figure 13: Test 1a	62
Figure 14: Test 1a	63
Figure 15: Test 1b	64
Figure 16: Test 1b	64
Figure 17: Test 2a	66
Figure 18: Test 2a	66
Figure 19: Test 2a	67
Figure 20: Test 2b	68
Figure 21: Test 2b	68
Figure 22: Test 2b	69
Figure 23: Test 3a	70
Figure 24: Test 3a	70
Figure 25: Test 3a	71
Figure 26: Test 3b	72
Figure 27: Test 3c	73
Figure 28: Test 3d	74
Figure 29: Test 3d	74
Figure 30: Test 3d	75
Figure 31: Test 4a	76
Figure 32: Test 4a	76
Figure 33: Test 4a	77
Figure 34: Test 4b	78
Figure 35: Test 4b	78
Figure 36: Test 4c	79
Figure 37: Test 4c	80

Figure 38: Test 5a	81
Figure 39: Test 5b	82
Figure 40: Syntax Error	83
Figure 41: Syntax Error	84
Figure 42: Syntax Error	84
Figure 43: Runtime NPE in “Read File” Dialog	85
Figure 44: Runtime NPE in “Read File” Dialog	85
Figure 45: Logical Refund Calculation (Standard Cancel)	87
Figure 46: Logical Refund Calculation (Standard Cancel)	87

Table of Tables

Table 1: Method Descriptions 1	37
Table 2: Method Descriptions 2	37
Table 3: Method Descriptions 3	38
Table 4: Method Descriptions 4	38
Table 5: Method Descriptions 5	38
Table 6: Method Descriptions 6	39
Table 7: Method Descriptions 7	39
Table 8: Method Descriptions 8	39
Table 9: Method Descriptions 9	40
Table 10: Method Descriptions 10	40
Table 11: Method Descriptions 11	41
Table 12: Method Descriptions 12	41
Table 13: Method Descriptions 13	41
Table 14: Method Descriptions 14	42
Table 15: Method Descriptions 15	42
Table 16: Method Descriptions 16	43
Table 17: Method Descriptions 17	43
Table 18: Method Descriptions 18	44
Table 19: Method Descriptions 19	44
Table 20: Method Descriptions 20	45
Table 21: Method Descriptions 21	45
Table 22: Method Descriptions 22	45
Table 23: Method Descriptions 23	46
Table 24: Method Descriptions 24	46
Table 25: Method Descriptions 25	47
Table 26: Method Descriptions 26	47
Table 27: Method Descriptions 27	48
Table 28: Method Descriptions 28	48
Table 29: Method Descriptions 29	49
Table 30: Method Descriptions 30	49
Table 31: Method Descriptions 31	50
Table 32: Method Descriptions 32	50
Table 33: Method Descriptions 33	51
Table 34: Method Descriptions 34	51
Table 35: Method Descriptions 35	52
Table 36: Method Descriptions 36	52
Table 37: Method Descriptions 37	53

Table 38: Method Descriptions 38	53
Table 39: Method Descriptions 39	54
Table 40: Method Descriptions 40	54
Table 41: Method Descriptions 41	55
Table 42: Method Descriptions 42	55
Table 43: Method Descriptions 43	56
Table 44: Method Descriptions 44	56
Table 45: Method Descriptions 45	57
Table 46: Method Descriptions 46	57
Table 47: Method Descriptions 47	57
Table 48: Method Descriptions 48	58
Table 49: Method Descriptions 49	58
Table 50: Method Descriptions 50	59
Table 51: Method Descriptions 51	59
Table 52: Method Descriptions 52	60
Table 53: Method Descriptions 53	60
Table 54: Method Descriptions 54	61
Table 55: Method Descriptions 55	61
Table 56: Test 1a	62
Table 57: Test 1b	64
Table 58: Test 2a	65
Table 59: Test 2b	68
Table 60: Test 3a	69
Table 61: Test 3b	71
Table 62: Test 3c	72
Table 63: Test 3d	73
Table 64: Test 4a	75
Table 65: Test 4b	77
Table 66: Test 4c	79
Table 67: Test 5a	80
Table 68: Test 5b	81
Table 69: Syntax Error	83
Table 70: Runtime NPE in "Read File" Dialog	85
Table 71: Logical Refund Calculation (Standard Cancel)	86

1. Introduction

Imagine writing a code once and viewing it run on Windows, macOS, or Linux without a problem courtesy to Java's "Write once, run everywhere" approach with the help of JVM. Java is a high-level, object-oriented programming language which helps to write scalable codes to reduce redundancy. This project is focusing on developing a program along the principles of OOP and UI/UX development to solve a real-world issue. Here in this coursework, we will create visitor classes and various methods like logging visits, buying products, canceling purchases, and other specific features depending on the visitor types. In other words, it is to track and manage visitors, arts, and their purchases. And by using swing we build UI components like panels, containers, and buttons. The final thing is to make a cross-platform, easy to maintain Art Gallery Management system with clear structure and interface.

2. Aims and Objectives

- Clean Class Breakdown: We will divide the program into classes (ArtGalleryVisitor, StandardVisitor, EliteVisitor) using OOP principles like encapsulation, inheritance and learn it.
- Basic Swing Interface: We will create a simple UI using frames, panels, and buttons for Log Visit, Buy, Cancel Purchase, and View Bill.
- Real-Life Gallery Logic: In the backend the program will block any purchase until someone has logged a visit, calculate discount and allocate reward points automatically, apply return fees and clarify limits, and display the bill.
- Runs Everywhere & Stays Tidy: Because it is Java, the same system should work in any OS. We will use meaningful names and write comments so in future any individual can make changes, fix a bug, and add a feature without scouring the whole program.

3. GUI Wireframes

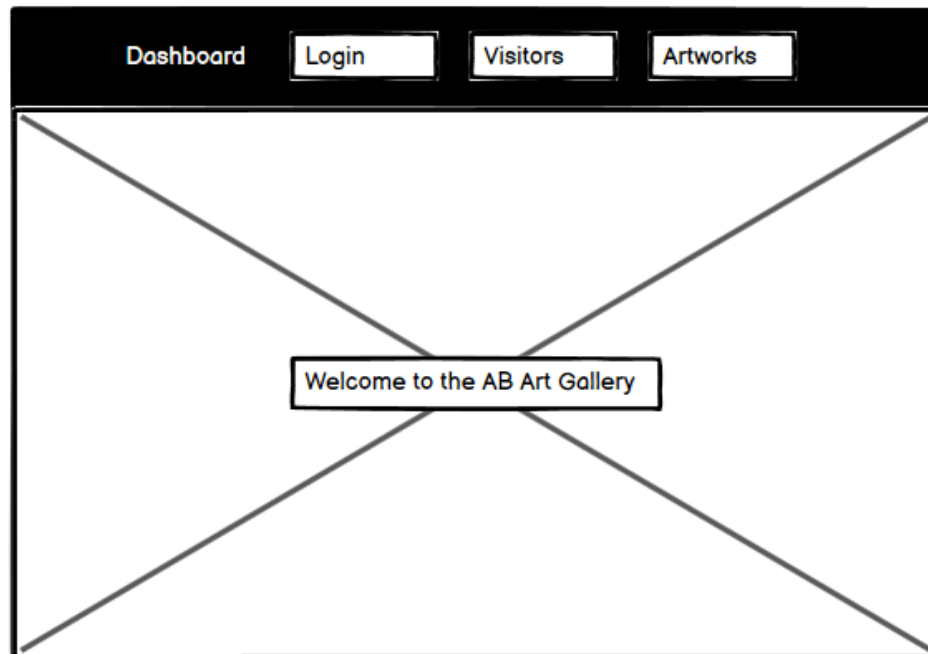


Figure 1: Wireframe Dashboard

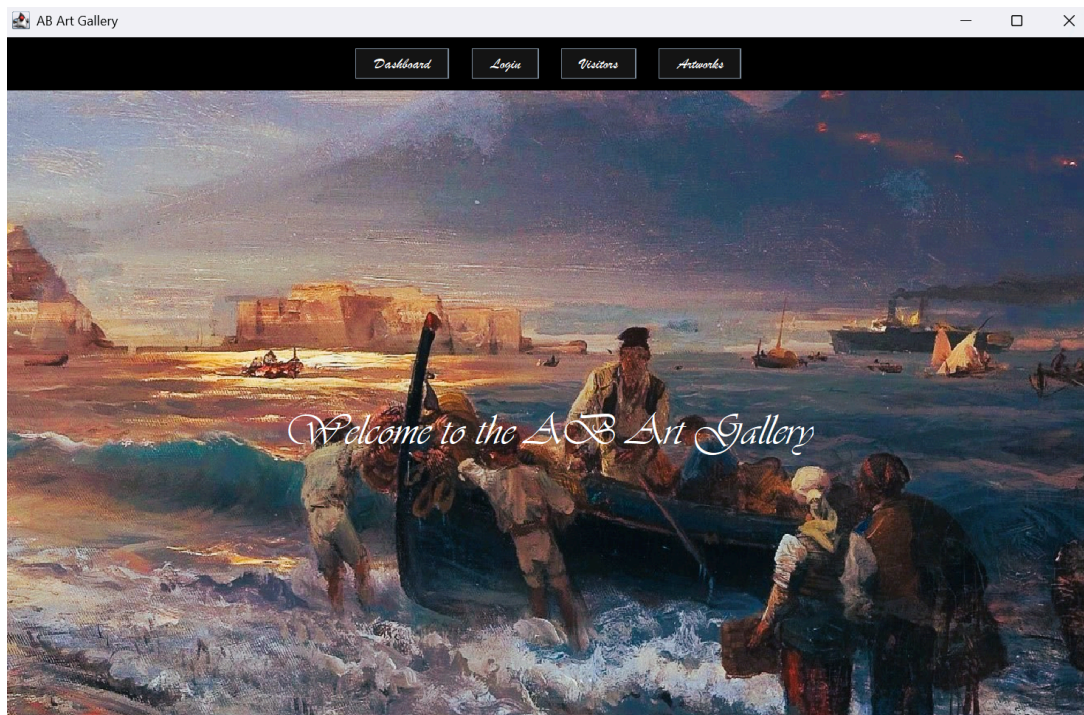
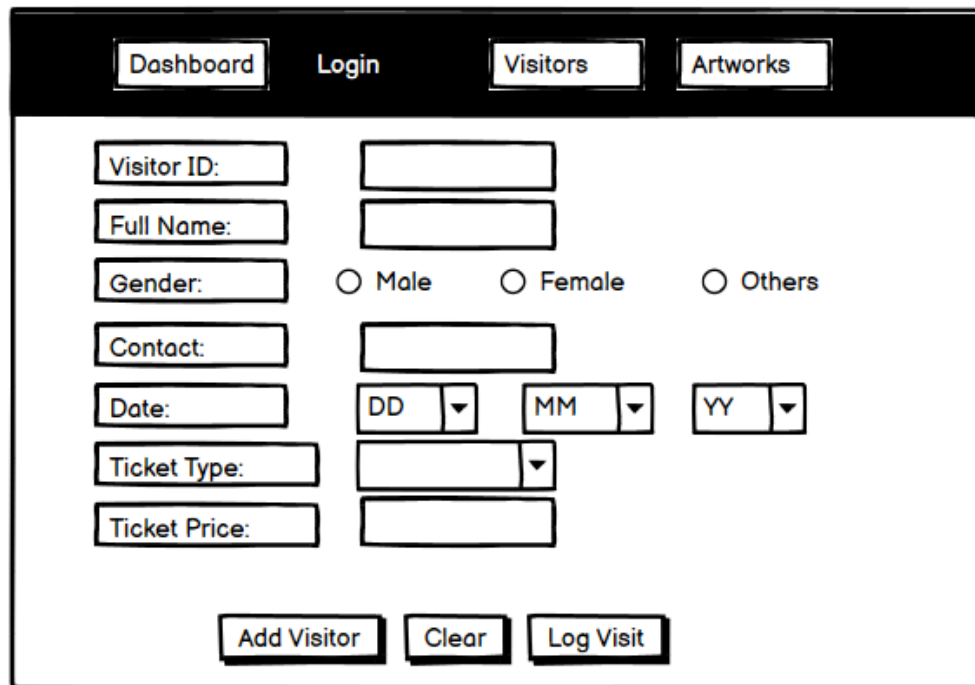
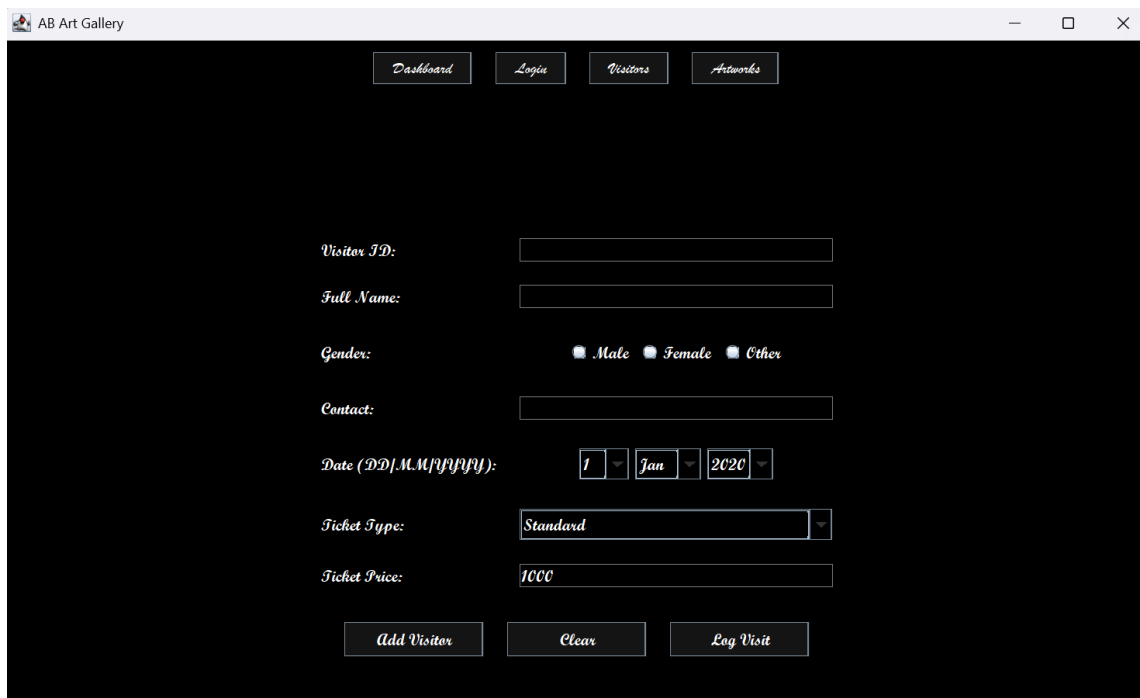


Figure 2: GUI Dashboard



A wireframe login form for an application. At the top, there is a navigation bar with four buttons: 'Dashboard', 'Login', 'Visitors', and 'Artworks'. Below this, the form contains several input fields and controls arranged in two columns. The left column contains labels for 'Visitor ID:', 'Full Name:', 'Gender:', 'Contact:', 'Date:', 'Ticket Type:', and 'Ticket Price:'. The right column contains corresponding input fields: a text box for Visitor ID, a text box for Full Name, radio buttons for Gender (Male, Female, Others), a text box for Contact, date dropdowns for DD, MM, and YY, a dropdown for Ticket Type, and a text box for Ticket Price. At the bottom of the form, there are three buttons: 'Add Visitor', 'Clear', and 'Log Visit'.

Figure 3: Wireframe Login



A screenshot of the final GUI login form for 'AB Art Gallery'. The window title is 'AB Art Gallery'. The navigation bar at the top has buttons for 'Dashboard', 'Login', 'Visitors', and 'Artworks'. The form fields are as follows: 'Visitor ID:' with a text box; 'Full Name:' with a text box; 'Gender:' with radio buttons for 'Male', 'Female', and 'Other'; 'Contact:' with a text box; 'Date (DD/MM/YYYY):' with dropdowns for '1', 'Jan', and '2020'; 'Ticket Type:' with a dropdown menu showing 'Standard'; and 'Ticket Price:' with a text box showing '1000'. At the bottom, there are three buttons: 'Add Visitor', 'Clear', and 'Log Visit'.

Figure 4: GUI Login

The wireframe shows a web application interface for the 'Visitors' section. At the top, there are four navigation tabs: 'Dashboard', 'Login', 'Visitors' (which is highlighted), and 'Artworks'. Below the tabs is a table with the following headers: 'ID', 'Name', 'Gender', 'Contact', 'Date', 'Type', and 'Price'. The table body is currently empty. At the bottom of the page, there is a form area containing a label 'Visitor ID:' followed by an input field, and four buttons: 'Assign', 'Check', 'Clear', and 'Read'.

Figure 5: Wireframe Visitors

The GUI shows a web application window titled 'AB Art Gallery'. It features a navigation bar with four tabs: 'Dashboard', 'Login', 'Visitors' (highlighted), and 'Artworks'. The main content area is divided into two sections: 'Standard Visitors' and 'Elite Visitors'. The 'Standard Visitors' table has headers: 'ID', 'Name', 'Gender', 'Contact', 'Date', 'Type', 'Price', 'Visits', 'Upgrade?', and 'Discount...'. The 'Elite Visitors' table has headers: 'ID', 'Name', 'Gender', 'Contact', 'Date', 'Type', 'Price', 'Visits', 'Advisor', and 'Event Acc...'. Both tables are currently empty. At the bottom, there is a form area with a label 'Visitor ID:' followed by an input field, and four buttons: 'Assign Per...', 'Check Upg...', 'Clear', and 'Read File'.

Figure 6: GUI Visitors

The wireframe shows a navigation bar with four buttons: Dashboard, Login, Visitors, and Artworks. The Artworks section contains four input fields labeled Visitor ID, Artwork Name, Price, and Cancel Reason. Below these are six buttons: Buy Artwork, Cancel Artwork, Generate Bill, Clear, Calculate Discount, and Calculate Reward. At the bottom is a table with three columns: ID, Artwork Name, and Price.

ID	Artwork Name	Price
----	--------------	-------

Figure 7: Wireframe Artworks

The GUI is a dark-themed version of the wireframe. It features a title bar with the text 'AB Art Gallery' and standard window controls. The navigation bar has buttons for Dashboard, Login, Visitors, and Artworks. The input fields are labeled 'Visitor ID:', 'Artwork Name:', 'Price:', and 'Cancel Reason:'. The buttons are labeled 'Buy Artwork', 'Cancel Artwork', 'Generate Bill', 'Clear', 'Calculate Reward Points', and 'Calculate Discount'. The table at the bottom has columns labeled 'ID', 'Artwork Name', and 'Price'.

ID	Artwork Name	Price
----	--------------	-------

Figure 8: GUI Artworks

4. Class Diagram

4.1. ArtGalleryDiagram Class Diagram

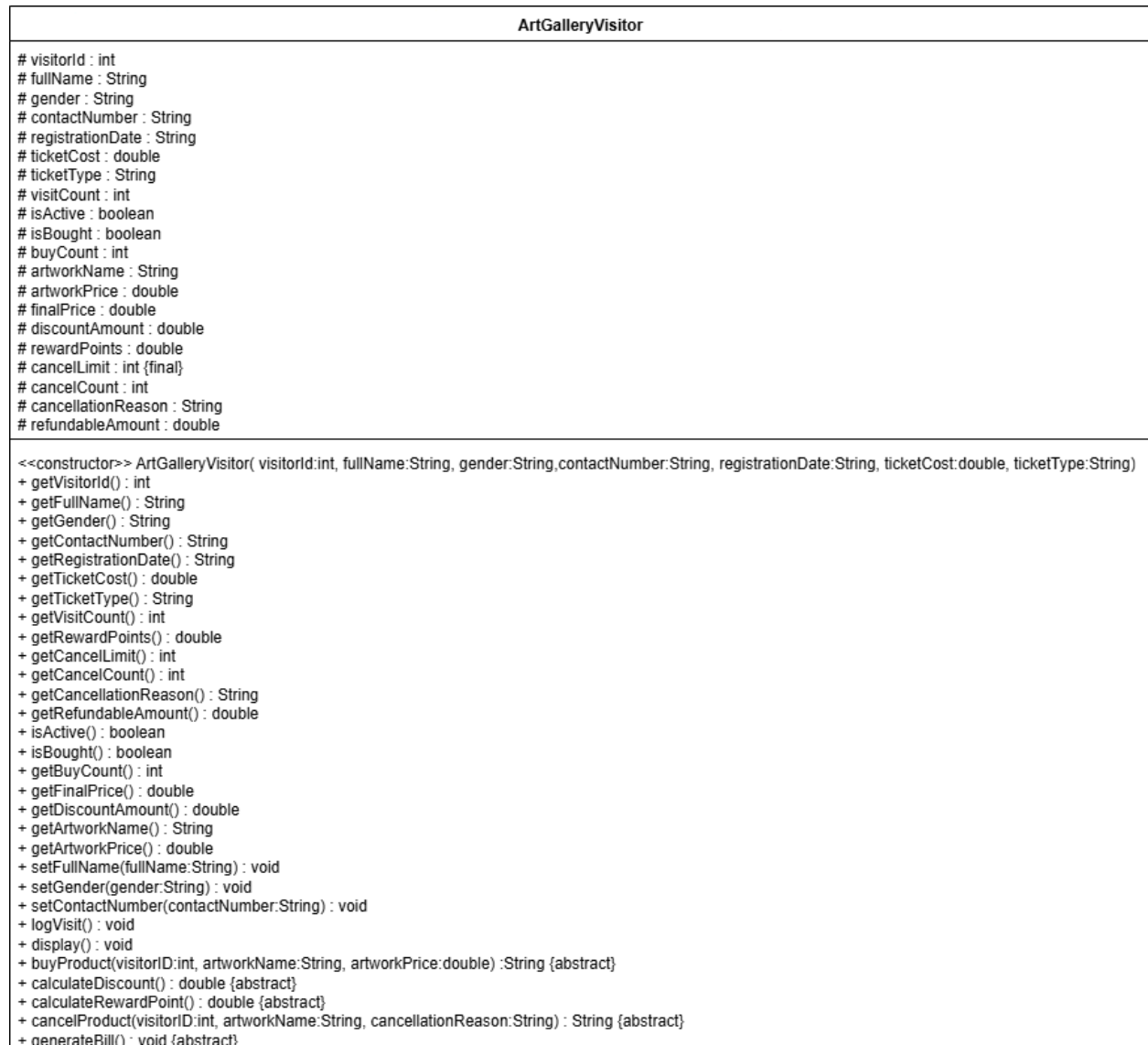


Figure 9: ArtGalleryVisitor Class Diagram

4.2. StandardVisitor Class Diagram

StandardVisitor
- isEligibleForDiscountUpgrade : boolean - visitLimit : int {final} = 5 - discountPercent : float
<<constructor>> StandardVisitor(visitorId:int, fullName:String, gender:String, contactNumber:String, registrationDate:String, ticketCost:double, ticketType:String) + isEligibleForDiscountUpgrade() : boolean + getVisitLimit() : int + getDiscountPercent() : float + checkDiscountUpgrade() : boolean + buyProduct(visitorID:int, artworkName:String, artworkPrice:double) : String + calculateDiscount() : double + calculateRewardPoint() : double + cancelProduct(visitorID:int, artworkName:String, cancellationReason:String) : String + generateBill() : void + display() : void - terminateVisitor() : void

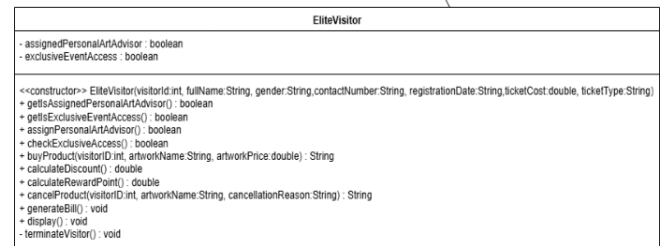
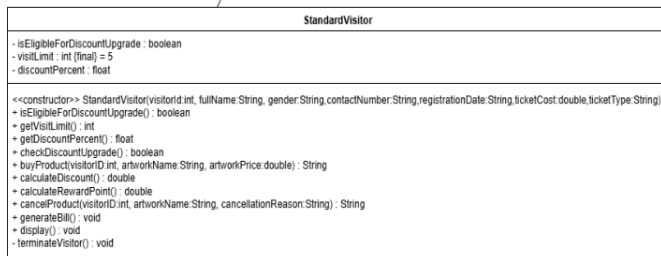
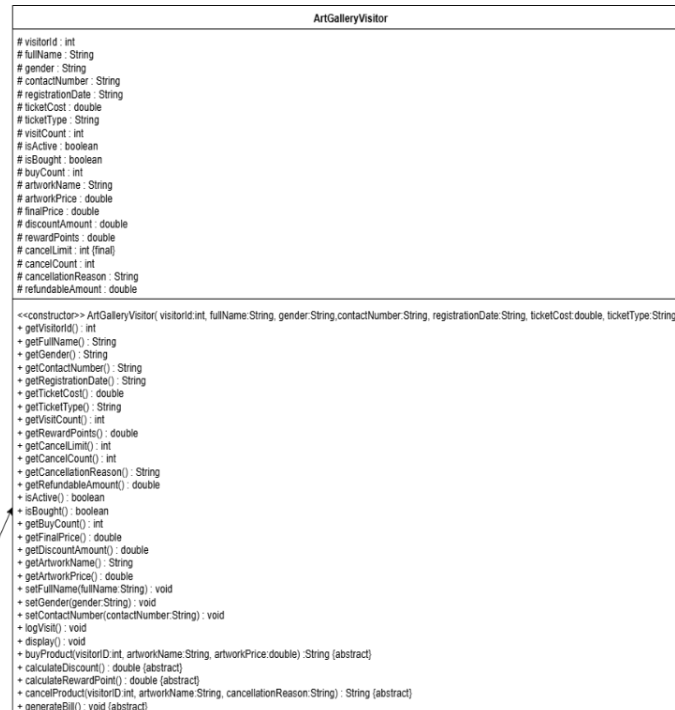
Figure 10: StandardVisitor Class Diagram

4.3. EliteVisitor Class Diagram

EliteVisitor
- assignedPersonalArtAdvisor : boolean - exclusiveEventAccess : boolean
<<constructor>> EliteVisitor(visitorId:int, fullName:String, gender:String, contactNumber:String, registrationDate:String, ticketCost:double, ticketType:String) + getIsAssignedPersonalArtAdvisor() : boolean + getIsExclusiveEventAccess() : boolean + assignPersonalArtAdvisor() : boolean + checkExclusiveAccess() : boolean + buyProduct(visitorID:int, artworkName:String, artworkPrice:double) : String + calculateDiscount() : double + calculateRewardPoint() : double + cancelProduct(visitorID:int, artworkName:String, cancellationReason:String) : String + generateBill() : void + display() : void - terminateVisitor() : void

Figure 11: EliteVisitor Class Diagram

4.4. Final Class Diagram



5. Pseudocode

5.1. NavBar Button

```
ON CLICK NAV_BUTTON(page)

    CALL cardLayout.SHOW(mainPanel, page)

END ON
```

5.2. Ticket Type Combo - Login

```
ON ITEM STATE CHANGED cbType

    IF STATE == SELECTED THEN

        IF SELECTED ITEM == "Elite" THEN

            SET txtPrice.TEXT ← "2000"

        ELSE

            SET txtPrice.TEXT ← "1000"

        END IF

    END IF

END ON
```

5.3. Add Visitor - Login

```
ON CLICK btnAdd

    TRY

        SET idStr ← TRIM(txtId.TEXT)

        SET name ← TRIM(txtName.TEXT)

        SET contact ← TRIM(txtContact.TEXT)
```

IF idStr EMPTY **OR** name EMPTY **OR** contact EMPTY **OR** genderGroup
UNSELECTED **THEN**

SHOW WARNING "Please fill in all required fields."

RETURN

END IF

FOR EACH ch **IN** idStr

IF ch **NOT** DIGIT **THEN**

THROW "Visitor ID must contain only numbers."

END IF

END FOR

FOR EACH ch **IN** name

IF ch **IS** DIGIT **THEN**

THROW "Full Name must not contain numbers."

END IF

END FOR

FOR EACH ch **IN** contact

IF ch **NOT** DIGIT **THEN**

THROW "Contact number must contain only numbers."

END IF

END FOR

IF LENGTH(contact) != 10 **THEN**

 THROW "Contact number must be exactly 10 digits."

END IF

SET id ← PARSE INT(idStr)

SET duplicate ← **FALSE**

FOR EACH v **IN** visitorList

IF v.getVisitorId() == id **THEN**

SET duplicate ← **TRUE**

 BREAK

END IF

END FOR

IF duplicate **THEN**

SHOW WARNING "Visitor ID already exists."

RETURN

END IF

SET gender ← SELECTED TEXT(genderGroup)

SET date ← cbDay.VAL + "/" + cbMonth.VAL + "/" + cbYear.VAL

SET type ← cbType.VAL

IF type == "Elite" **THEN**

SET ticketCost ← 2000

```
        CREATE visitor ← EliteVisitor(id, name, gender, contact, date, ticketCost, type)

    ELSE

        SET ticketCost ← 1000

        CREATE visitor ← StandardVisitor(id, name, gender, contact, date, ticketCost,
type)

    END IF

    APPEND visitor TO visitorList

    SET lastVisitorType ← type

    SHOW INFO "Visitor added successfully!"

CATCH IllegalArgumentException ex

    SHOW ERROR ex.MESSAGE

CATCH Exception ex

    SHOW ERROR "Error: " + ex.MESSAGE

END TRY

END ON
```

5.4. Clear - Login

```
ON CLICK btnClear

    SET txtId.TEXT ← ""

    SET txtName.TEXT ← ""

    SET txtContact.TEXT ← ""

    CLEAR SELECTION genderGroup
```

SET cbDay.**SELECT** $\leftarrow$ 0

SET cbMonth.**SELECT** $\leftarrow$ 0

SET cbYear.**SELECT** $\leftarrow$ 0

SET cbType.**SELECT** $\leftarrow$ 0

SET txtPrice.**TEXT** $\leftarrow$ "1000"

END ON

5.5. Log Visit - Login

ON CLICK btnLogVisit

SET idStr $\leftarrow$ TRIM(txtId.TEXT)

SET name $\leftarrow$ TRIM(txtName.TEXT)

SET contact $\leftarrow$ TRIM(txtContact.TEXT)

IF idStr EMPTY **OR** name EMPTY **OR** contact EMPTY **OR** genderGroup
UNSELECTED **THEN**

SHOW WARNING "Please fill in all required fields."

RETURN

END IF

TRY

VALIDATE idStr **ALL** DIGITS

VALIDATE name HAS **NO** DIGITS

VALIDATE contact **ALL** DIGITS

IF LENGTH(contact) != 10 **THEN**

THROW "Contact number must be exactly 10 digits."

END IF

SET id ← **PARSE INT**(idStr)

SET found ← **FALSE**

FOR EACH visitor **IN** visitorList

IF visitor.getVisitorId() == id **THEN**

CALL visitor.logVisit()

SHOW INFO "Visit logged. Total visits: " + visitor.getVisitCount()

SET found ← **TRUE**

BREAK

END IF

END FOR

IF NOT found **THEN**

SHOW ERROR "Visitor ID not found."

END IF

CATCH IllegalArgumentException ex

SHOW ERROR ex.MESSAGE

CATCH Exception ex

SHOW ERROR "Error: " + ex.MESSAGE

END TRY

END ON

5.6. Clear – Visitors

ON CLICK clearBtn

SET idField.TEXT $\leftarrow$ ""

END ON

5.7. Assign Advisor (ELITE) – Visitors

ON CLICK assignAdvisorBtn

SET s $\leftarrow$ TRIM(idField.TEXT)

IF s EMPTY **THEN**

SHOW ERROR "Enter Visitor ID."

RETURN

END IF

TRY

SET id $\leftarrow$ PARSE INT(s)

SET found $\leftarrow$ FALSE

FOR EACH v **IN** visitorList

IF v.getVisitorId() == id **THEN**

SET found $\leftarrow$ **TRUE**

IF v INSTANCEOF EliteVisitor **THEN**

SET ev $\leftarrow$ **CAST** v **AS** EliteVisitor

```
        SET assigned ← ev.assignPersonalArtAdvisor()

        CALL ev.checkExclusiveAccess()

        IF assigned THEN

            SHOW INFO "Advisor assigned or already eligible."

        ELSE

            SHOW INFO "Not enough reward points."

        END IF

        CALL REFRESH TABLES()

    ELSE

        SHOW ERROR "This visitor is not an Elite Visitor."

    END IF

    BREAK

END IF

END FOR

IF NOT found THEN

    SHOW ERROR "Visitor ID not found."

END IF

CATCH NumberFormatException

    SHOW ERROR "Visitor ID must be a number."

CATCH Exception ex

    SHOW ERROR "Unexpected error: " + ex.MESSAGE

END TRY

END ON
```

5.8. Check Upgrade (STANDARD) – Visitors

ON CLICK checkUpgradeBtn

SET s ← TRIM(idField.TEXT)

IF s EMPTY **THEN**

SHOW ERROR "Enter Visitor ID."

RETURN

END IF

TRY

SET id ← PARSE INT(s)

SET found ← **FALSE**

FOR EACH v **IN** visitorList

IF v.getVisitorId() == id **THEN**

SET found ← **TRUE**

IF v INSTANCEOF StandardVisitor **THEN**

SET sv ← **CAST** v **AS** StandardVisitor

SET eligible ← sv.checkDiscountUpgrade()

IF eligible **THEN**

SHOW INFO "Eligible for 15% discount."

ELSE

SHOW INFO "Visit limit not reached (requires 5)."

END IF

CALL REFRESH TABLES()

```
        ELSE

            SHOW ERROR "This visitor is not a Standard Visitor."

        END IF

        BREAK

    END IF

END FOR

IF NOT found THEN

    SHOW ERROR "Visitor ID not found."

END IF

CATCH NumberFormatException

    SHOW ERROR "Visitor ID must be a number."

CATCH Exception ex

    SHOW ERROR "Unexpected error: " + ex.MESSAGE

END TRY

END ON
```

5.9. Read File – Visitors

```
ON CLICK readFileBtn

    SET f ← FILE("visitors.txt")

    IF NOT EXISTS(f) THEN

        SHOW WARNING "visitors.txt not found."

        RETURN

    END IF
```

CREATE stdFileModel **WITH** STANDARD COLUMNS

CREATE eliteFileModel **WITH** ELITE COLUMNS

SET inStd $\leftarrow$ **FALSE**

SET inElite $\leftarrow$ **FALSE**

SET skip $\leftarrow$ 0

TRY

OPEN br **FOR** READ f

WHILE READ LINE(br) **INTO** line

IF STARTS **WITH**(line, "==== Standard Visitors") **THEN**

SET inStd $\leftarrow$ **TRUE**

SET inElite $\leftarrow$ **FALSE**

SET skip $\leftarrow$ 2

CONTINUE

END IF

IF STARTS **WITH**(line, "==== Elite Visitors") **THEN**

SET inElite $\leftarrow$ **TRUE**

SET inStd $\leftarrow$ **FALSE**

SET skip $\leftarrow$ 2

CONTINUE

END IF

IF skip > 0 **THEN**

DECREMENT skip

```
        CONTINUE

    END IF

    IF line EMPTY OR STARTS WITH(line, "-") OR STARTS WITH(line, "(none)")
THEN
        CONTINUE

    END IF

    SET parts ← SPLIT(line BY 2+ SPACES)

    IF inStd AND LENGTH(parts) ≥ 10 THEN

        ADD ROW stdFileModel WITH parts[0..9]

    ELSE IF inElite AND LENGTH(parts) ≥ 10 THEN

        ADD ROW eliteFileModel WITH parts[0..9]

    END IF

END WHILE

CLOSE br

CATCH Exception ex

    SHOW ERROR "Error reading visitors.txt: " + ex.MESSAGE

RETURN

END TRY

BUILD MODAL DIALOG WITH TABLE(stdFileModel) AND TABLE(eliteFileModel)

SHOW DIALOG

END ON
```

5.10. Buy Artwork – Artworks

ON CLICK btnBuy

SET idText $\leftarrow$ TRIM(txtVisitorId.TEXT)

SET nameText $\leftarrow$ TRIM(txtName.TEXT)

SET priceText $\leftarrow$ TRIM(txtPrice.TEXT)

IF idText EMPTY **OR** nameText EMPTY **OR** priceText EMPTY **THEN**

SHOW ERROR "Enter Visitor ID, Artwork name, and price."

RETURN

END IF

TRY

VALIDATE idText **ALL** DIGITS

VALIDATE nameText **LETTERS OR** SPACE

SET price $\leftarrow$ PARSE DOUBLE(priceText)

IF price $\leq$ 0 **THEN**

THROW "Price must be greater than 0."

END IF

SET visitorId $\leftarrow$ PARSE INT(idText)

FIND target **IN** visitorList **WHERE** target.getVisitorId() == visitorId

IF target **IS NULL THEN**

SHOW ERROR "Visitor ID not found."

RETURN

END IF

SET result $\leftarrow$ target.buyProduct(visitorId, nameText, price)

SHOW INFO result

CATCH NumberFormatException

SHOW ERROR "Visitor ID and Price must be numeric."

CATCH IllegalArgumentException ex

SHOW ERROR ex.MESSAGE

CATCH Exception ex

SHOW ERROR "Unexpected error: " + ex.MESSAGE

END TRY

END ON

5.11. Cancel Artwork – Artworks

ON CLICK btnCancel

SET idText $\leftarrow$ TRIM(txtVisitorId.TEXT)

SET nameText $\leftarrow$ TRIM(txtName.TEXT)

SET priceText $\leftarrow$ TRIM(txtPrice.TEXT)

SET reason $\leftarrow$ TRIM(txtCancelReason.TEXT)

IF idText EMPTY **OR** nameText EMPTY **OR** priceText EMPTY **OR** reason EMPTY
THEN

SHOW ERROR "Please fill all fields."

RETURN

END IF

TRY

VALIDATE idText **ALL** DIGITS

VALIDATE nameText **LETTERS OR** SPACE

SET price $\leftarrow$ PARSE DOUBLE(priceText)

IF price $\leq$ 0 **THEN**

THROW "Price must be a positive number."

END IF

SET visitorId $\leftarrow$ PARSE INT(idText)

SET found $\leftarrow$ **FALSE**

FOR EACH v **IN** visitorList

IF v.getVisitorId() == visitorId **THEN**

SET found $\leftarrow$ **TRUE**

IF NOT v.isBought() **THEN**

THROW "This visitor has not purchased any artwork."

END IF

IF EQUALS IGNORE **CASE**(nameText, v.getArtworkName()) **AND** price ==
v.getArtworkPrice() **THEN**

SET msg $\leftarrow$ v.cancelProduct(visitorId, nameText, reason)

```
CLEAR TABLE ROWS(model)

SHOW WARNING msg

RETURN

ELSE

    THROW "No matching artwork found for given name and price."

END IF

END IF

END FOR

IF NOT found THEN

    THROW "Visitor ID not found."

END IF

CATCH NumberFormatException

    SHOW ERROR "Visitor ID and Price must be valid numbers."

CATCH IllegalArgumentException ex

    SHOW ERROR ex.MESSAGE

CATCH Exception ex

    SHOW ERROR "Unexpected error: " + ex.MESSAGE

END TRY

END ON

5.12. Generate Bill – Artworks

ON CLICK btnBill

    SET idText ← TRIM(txtVisitorId.TEXT)

    IF idText EMPTY THEN
```

SHOW ERROR "Enter Visitor ID."

RETURN

END IF

TRY

VALIDATE idText **ALL** DIGITS

SET visitorId ← PARSE INT(idText)

FIND target **IN** visitorList **WHERE** target.getVisitorId() == visitorId

IF target **IS NULL THEN**

THROW "Visitor ID not found."

END IF

IF NOT target.isBought() **THEN**

THROW "This visitor has not purchased any artwork."

END IF

SET discount ← target.calculateDiscount()

SET finalAmount ← target.getArtworkPrice() - discount

PROMPT SAVE FILE **DEFAULT** "Invoice_<visitorId>.txt"

IF USER CANCELLED THEN

RETURN

END IF

WRITE INVOICE LINES (ID, NAME, TICKET, ARTWORK, PRICE, DISCOUNT, FINAL)

SHOW INFO "Invoice generated successfully."

CATCH NumberFormatException

SHOW ERROR "Visitor ID must be numeric."

CATCH IllegalArgumentException ex

SHOW ERROR ex.MESSAGE

CATCH Exception ex

SHOW ERROR "Error writing invoice: " + ex.MESSAGE

END TRY

END ON

5.13. Clear – Artworks

ON CLICK btnClear

SET txtVisitorId.TEXT ← ""

SET txtName.TEXT ← ""

SET txtPrice.TEXT ← ""

SET txtCancelReason.TEXT ← ""

END ON

5.14. Calculate Discount – Artworks

ON CLICK btnDiscount

SET idText ← TRIM(txtVisitorId.TEXT)

IF idText EMPTY **THEN**

SHOW ERROR "Please enter Visitor ID."

RETURN

END IF

TRY

SET id ← PARSE INT(idText)

SET found ← **FALSE**

FOR EACH visitor **IN** visitorList

IF visitor.getVisitorId() == id **THEN**

SET found ← **TRUE**

IF NOT visitor.isBought() **THEN**

THROW "This visitor has not purchased any artwork yet."

END IF

SET discount ← visitor.calculateDiscount()

SHOW INFO "Discount: Rs. " + discount

BREAK

END IF

END FOR

IF NOT found **THEN**

SHOW ERROR "Visitor ID not found."

END IF

CATCH NumberFormatException

```
    SHOW ERROR "Visitor ID must be a number."

CATCH IllegalArgumentException ex

    SHOW ERROR ex.MESSAGE

CATCH Exception ex

    SHOW ERROR "Unexpected error: " + ex.MESSAGE

END TRY

END ON
```

5.15. Calculate Reward – Artworks

```
ON CLICK btnReward

    SET idText ← TRIM(txtVisitorId.TEXT)

    IF idText EMPTY THEN

        SHOW ERROR "Please enter Visitor ID."

        RETURN

    END IF
```

```
TRY

    SET id ← PARSE INT(idText)

    SET found ← FALSE

    FOR EACH visitor IN visitorList

        IF visitor.getVisitorId() == id THEN

            SET found ← TRUE

            IF NOT visitor.isBought() THEN
```

```
        THROW "This visitor has not purchased any artwork yet."

    END IF

    SET reward ← visitor.calculateRewardPoint()

    SHOW INFO "Reward Points: " + reward

    BREAK

END IF

END FOR

IF NOT found THEN

    SHOW ERROR "Visitor ID not found."

END IF

CATCH NumberFormatException

    SHOW ERROR "Visitor ID must be a number."

CATCH IllegalArgumentException ex

    SHOW ERROR ex.MESSAGE

CATCH Exception ex

    SHOW ERROR "Unexpected error: " + ex.MESSAGE

END TRY

END ON
```

6. Method Descriptions

6.1. ArtGalleryVisitor Methods

Method Signature	<code>public ArtGalleryVisitor(int visitorId, String fullName, String gender, String contactNumber, String registrationDate, double ticketCost, String ticketType)</code>
Method Description	Constructs the base visitor entity and prepares all state for safe downstream operations such as visit logging, purchasing, cancelling, and billing. This constructor centralizes the one-time initialization of identity (ID, name, gender, contact), registration metadata (date), and ticketing (cost, type). A predictable baseline avoids accidental reuse of stale values and lets subclasses rely on consistent defaults.
Parameters	visitorId: unique integer ID used as primary key; fullName: complete display name; gender: label (Male/Female/Other); contactNumber: validated phone string; registrationDate: sign-up date string; ticketCost: base ticket price; ticketType: Standard/Elite.
Return Type	None
Logic	Copies parameters into protected fields so subclasses can read them without breaking encapsulation. Initializes counters visitCount=0, buyCount=0, cancelCount=0, rewardPoints=0 so analytics start from zero.

	Resets purchase context to a neutral state (artworkName=null, artworkPrice=0, discountAmount=0, finalPrice=0, refundableAmount=0). Sets status flags (isActive=false, isBought=false) to require an explicit visit before financial flows. Leaves policy constant cancelLimit=3. This makes every subsequent method (discount, rewards, cancel, bill) run on known good state.
--	--

Table 1: Method Descriptions 1

Method Signature	public int getVisitorId()
Method Description	Retrieves the immutable identifier assigned at construction. This value is used throughout the GUI and business logic as the authoritative lookup key.
Parameters	—
Return Type	int
Logic	Returns the stored visitorId directly without transformation. No recalculation is performed because identity never changes within an instance.

Table 2: Method Descriptions 2

Method Signature	public String getFullName()
Method Description	Returns the current full name for display on tables, dialogs, and invoices. May reflect user corrections after registration.
Parameters	—
Return Type	String
Logic	Simply returns fullName. Formatting/validation (e.g., trimming, capitalization) stays in the UI to keep the model lean.

Table 3: Method Descriptions 3

Method Signature	public String getGender()
Method Description	Exposes the gender label captured at sign-up. Useful for reporting and UI filters.
Parameters	—
Return Type	String
Logic	Returns gender exactly as stored; the model avoids enforcing vocabulary so localization can remain in the UI.

Table 4: Method Descriptions 4

Method Signature	public String getContactNumber()
Method Description	Provides the contact number for receipts, support, or follow-ups.
Parameters	—
Return Type	String
Logic	Returns contactNumber without reformatting. Upstream validation guarantees 10 digits, keeping single source of truth.

Table 5: Method Descriptions 5

Method Signature	public String getRegistrationDate()
Method Description	Returns the original registration date string. Often printed on invoices or used in cohort analysis.
Parameters	—
Return Type	String
Logic	Returns registrationDate verbatim; date parsing/formatting lives in presentation layer to separate concerns.

Table 6: Method Descriptions 6

Method Signature	public double getTicketCost()
Method Description	Obtains the base ticket cost tied to this visitor's account.
Parameters	—
Return Type	double
Logic	Returns ticketCost; no caching or recomputation is needed because cost is stable once set.

Table 7: Method Descriptions 7

Method Signature	public String getTicketType()
Method Description	Provides the ticket category (Standard/Elite), enabling tier-specific rules elsewhere.
Parameters	—
Return Type	String
Logic	Returns ticketType directly so controllers can branch on tier without peeking into subclass internals.

Table 8: Method Descriptions 8

Method Signature	public int getVisitCount()
Method Description	Reports how many times the visitor has logged a visit. Used for upgrade logic and engagement metrics.
Parameters	—
Return Type	int
Logic	Returns visitCount. The only mutation path is logVisit(), which simplifies auditing.

Table 9: Method Descriptions 9

Method Signature	public double getRewardPoints()
Method Description	Shows the last computed reward points balance for the current purchase context. This value is calculated by subclass methods to reflect tier rules.
Parameters	—
Return Type	double
Logic	Returns rewardPoints as persisted, avoiding recomputation to prevent double-counting when multiple UI reads occur.

Table 10: Method Descriptions 10

Method Signature	public int getCancelLimit()
Method Description	Exposes the policy cap on cancellations so the UI can message limits consistently.
Parameters	—
Return Type	int
Logic	Returns the final constant cancelLimit (3). Keeping one definition avoids duplicated magic numbers.

Table 11: Method Descriptions 11

Method Signature	public int getCancelCount()
Method Description	Indicates how many successful cancellations have been recorded for this account.
Parameters	—
Return Type	int
Logic	Returns cancelCount. Only changes during cancelProduct() and termination, preserving an accurate policy state.

Table 12: Method Descriptions 12

Method Signature	public String getCancellationReason()
Method Description	Returns the last recorded cancellation reason for audit trails and user feedback.
Parameters	—
Return Type	String
Logic	Returns cancellationReason exactly as captured to ensure traceability in reports.

Table 13: Method Descriptions 13

Method Signature	public double getRefundableAmount()
Method Description	Returns the last computed refund value for the most recent cancellation.
Parameters	—
Return Type	double
Logic	Returns refundableAmount as set during cancelProduct(). No recalculation avoids accidental drift from what was communicated to the user.

Table 14: Method Descriptions 14

Method Signature	public boolean isActive()
Method Description	Indicates whether the visitor account has been activated by a logged visit. This acts as a prerequisite for purchasing/cancelling.
Parameters	—
Return Type	boolean
Logic	Returns isActive. Activation only occurs in logVisit() or is cleared via subclass termination routines.

Table 15: Method Descriptions 15

Method Signature	public boolean isBought()
Method Description	Signals whether there is a currently recorded purchase in this account context.
Parameters	—
Return Type	boolean
Logic	Returns isBought so dependent actions (discount, rewards, bill, cancel) can gate themselves safely.

Table 16: Method Descriptions 16

Method Signature	public int getBuyCount()
Method Description	Provides the number of recorded purchases historically. Useful for analytics and for verifying whether a billable purchase exists.
Parameters	—
Return Type	int
Logic	Returns buyCount. Even in a single-active-purchase model, this counter is informative for history and testing.

Table 17: Method Descriptions 17

Method Signature	public double getFinalPrice()
Method Description	Exposes the last computed payable amount after discount. Consumers should ensure discounts have been computed before reading.
Parameters	—
Return Type	double
Logic	Returns finalPrice. The model does not auto-recalculate here to avoid hidden side effects; callers control timing via calculateDiscount().

Table 18: Method Descriptions 18

Method Signature	public double getDiscountAmount()
Method Description	Returns the amount discounted from the artwork price. Displayed on bills and used for financial reconciliation.
Parameters	—
Return Type	double
Logic	Returns discountAmount. Mirrored with finalPrice so invoices reproduce exactly what was shown to the user.

Table 19: Method Descriptions 19

Method Signature	public String getArtworkName()
Method Description	Provides the name of the currently purchased artwork, if any.
Parameters	—
Return Type	String
Logic	Returns artworkName (may be null). Callers commonly check isBought() first to avoid null handling.

Table 20: Method Descriptions 20

Method Signature	public double getArtworkPrice()
Method Description	Provides the price of the current artwork, if any.
Parameters	—
Return Type	double
Logic	Returns artworkPrice. Zero typically indicates no active purchase in this design.

Table 21: Method Descriptions 21

Method Signature	public void setFullName(String fullName)
Method Description	Updates the stored full name when a correction or change is needed, keeping presentation data fresh.
Parameters	fullName: new name string
Return Type	void
Logic	Assigns this.fullName = fullName. Validation (no digits, non-empty) is the UI/controller's responsibility, keeping the model focused on state.

Table 22: Method Descriptions 22

Method Signature	public void setGender(String gender)
Method Description	Adjusts the gender label when the user updates their profile.
Parameters	gender: new label
Return Type	void
Logic	Writes gender to the field without transformation. Localization and vocabulary choices belong in the UI layer.

Table 23: Method Descriptions 23

Method Signature	public void setContactNumber(String contactNumber)
Method Description	Changes the contact number, e.g., when a user switches phone lines.
Parameters	contactNumber: validated phone string
Return Type	void
Logic	Sets this.contactNumber = contactNumber. Assumes the caller enforced 10-digit numeric format to maintain a single source of validation.

Table 24: Method Descriptions 24

Method Signature	public void logVisit()
Method Description	Registers a new gallery visit and flips the account to an active state. This is the gateway action that enables purchasing and cancellation for the session.
Parameters	—
Return Type	void
Logic	Increments visitCount by 1 to track engagement and sets isActive=true. Does not touch financial fields to ensure visiting is a side-effect-free action relative to pricing.

Table 25: Method Descriptions 25

Method Signature	public abstract String buyProduct(int visitorID, String artworkName, double artworkPrice)
Method Description	Abstract contract to begin a purchase. Subclasses implement tier-specific validations and side effects while ensuring identity safety.
Parameters	visitorID: claimed caller id; artworkName: chosen item; artworkPrice: price (>0)
Return Type	String
Logic	Implementations must: (1) verify visitorID == this.visitorId to prevent cross-account edits; (2) require isActive==true so a visit precedes a purchase; (3) enforce single-active-purchase semantics (reject duplicates); (4) validate positive price. On success, set artwork fields,

	isBought=true, buyCount++, and return a user-facing confirmation.
--	---

Table 26: Method Descriptions 26

Method Signature	public abstract double calculateDiscount()
Method Description	Abstract pricing routine that applies tier rules to derive a discount and compute a payable total used by rewards and billing.
Parameters	—
Return Type	double
Logic	If !isBought return 0 to avoid phantom discounts. Otherwise compute discountAmount from artworkPrice per tier, then set finalPrice = artworkPrice - discountAmount. Return discountAmount so callers can display it immediately while relying on cached finalPrice for subsequent steps.

Table 27: Method Descriptions 27

Method Signature	public abstract double calculateRewardPoint()
Method Description	Abstract reward routine that converts the post-discount payable amount into loyalty points according to tier rules.
Parameters	—
Return Type	double
Logic	If !isBought return 0. Ensure finalPrice is available (subclass may call calculateDiscount() if stale). Compute rewardPoints using multiplier (e.g., Standard ×5, Elite ×10) and persist it so repeated reads do not inflate totals.

Table 28: Method Descriptions 28

Method Signature	public abstract String cancelProduct(int visitorID, String artworkName, String cancellationReason)
Method Description	Abstract cancellation routine that reverses a purchase under policy constraints and maintains accurate audit fields.
Parameters	visitorID: claimed id; artworkName: must match current; cancellationReason: explanation text
Return Type	String
Logic	Validate identity and ensure there is an active purchase. Enforce cancelLimit (terminate if exceeded). Verify the artwork name matches to avoid cancelling the wrong item. Compute refundableAmount per tier policy; revoke or reset rewards as per per-purchase model; increment cancelCount, decrement buyCount; clear artwork and pricing fields; set isBought=false; return a clear status message.

Table 29: Method Descriptions 29

Method Signature	public abstract void generateBill()
Method Description	Abstract billing routine that produces a stable invoice view consistent with previously computed discount and total.
Parameters	—
Return Type	void
Logic	If a purchase exists, print a structured bill: identity, ticket, artwork, original price, discount

	amount, and final price. Avoid recomputation inside billing to ensure invoices match what the user already saw.
--	---

Table 30: Method Descriptions 30

Method Signature	public void display()
Method Description	Debug/diagnostic printout that captures a snapshot of all fields for grading, logging, or troubleshooting.
Parameters	—
Return Type	void
Logic	Sequentially prints identity, ticket info, visit history, status flags, current artwork/price, pricing fields (discountAmount, finalPrice), rewards, and cancellation state (counts, last reason, last refund).

Table 31: Method Descriptions 31

6.2. StandardVisitor Methods

Method Signature	public StandardVisitor(int visitorId, String fullName, String gender, String contactNumber, String registrationDate, double ticketCost, String ticketType)
Method Description	Initializes a Standard-tier account with a baseline 10% discount and a clear path to an upgraded 15% discount driven by visit frequency.
Parameters	(same parameters as base)
Return Type	None

Logic	Calls <code>super(...)</code> . Sets <code>discountPercent=0.10f</code> and <code>isEligibleForDiscountUpgrade=false</code> so everyone starts equally. Defines <code>visitLimit=5</code> to make upgrade deterministic and easy to message in the UI.
-------	--

Table 32: Method Descriptions 32

Method Signature	<code>public boolean getisEligibleForDiscountUpgrade()</code>
Method Description	Exposes whether the user has unlocked the 15% discount tier, allowing the UI to reflect the current status without recomputation.
Parameters	—
Return Type	<code>boolean</code>
Logic	Returns <code>isEligibleForDiscountUpgrade</code> exactly as last set by <code>checkDiscountUpgrade()</code> , avoiding hidden logic in getters.

Table 33: Method Descriptions 33

Method Signature	<code>public int getVisitLimit()</code>
Method Description	Returns the policy threshold (visits required) for the discount upgrade, keeping rule text centralized.
Parameters	—
Return Type	<code>int</code>
Logic	Returns 5, enabling tooltips and labels to derive from one authoritative source.

Table 34: Method Descriptions 34

Method Signature	public float getDiscountPercent()
Method Description	Reports the currently active Standard discount percentage so billing and badges can show accurate numbers.
Parameters	—
Return Type	float
Logic	Returns 0.10f or 0.15f depending on eligibility. Callers typically invoke checkDiscountUpgrade() before displaying.

Table 35: Method Descriptions 35

Method Signature	public boolean checkDiscountUpgrade()
Method Description	Evaluates visitCount against the policy threshold and promotes the account to the 15% tier when eligible; safe to call multiple times.
Parameters	—
Return Type	boolean
Logic	If visitCount >= visitLimit, sets isEligibleForDiscountUpgrade=true and discountPercent=0.15f; otherwise leaves values unchanged. Returns the resulting state for immediate UI messaging.

Table 36: Method Descriptions 36

Method Signature	<code>@Override public String buyProduct(int visitorID, String artworkName, double artworkPrice)</code>
Method Description	Performs all Standard purchase checks (identity, activity, duplication) and records a purchase cleanly.
Parameters	visitorID, artworkName, artworkPrice
Return Type	String
Logic	Require isActive and matching visitorID; reject duplicate of same artwork while an active purchase exists; reject non-positive price. On success, set artwork fields, isBought=true, buyCount++; return a clear confirmation.

Table 37: Method Descriptions 37

Method Signature	<code>@Override public double calculateDiscount()</code>
Method Description	Applies the Standard discount using the most recent tier (10% or 15%) and prepares finalPrice for rewards/billing.
Parameters	—
Return Type	double
Logic	If !isBought return 0; call checkDiscountUpgrade() to reflect new visits; set discountAmount=artworkPrice*discountPercent; finalPrice=artworkPrice-discountAmount; return discountAmount.

Table 38: Method Descriptions 38

Method Signature	@Override public double calculateRewardPoint()
Method Description	Computes per-purchase reward points for Standard from the current finalPrice, avoiding double counting on repeated reads.
Parameters	—
Return Type	double
Logic	If !isBought return 0; if finalPrice is stale, invoke calculateDiscount(); set rewardPoints = finalPrice * 5; return rewardPoints.

Table 39: Method Descriptions 39

Method Signature	@Override public String cancelProduct(int visitorID, String artworkName, String cancellationReason)
Method Description	Reverses a Standard purchase with policy enforcement (cap, name check) and correct financial resets.
Parameters	visitorID, artworkName, cancellationReason
Return Type	String
Logic	Validate visitorID; if cancelCount >= cancelLimit, terminateVisitor() and inform the user; if not bought, return error; ensure provided name matches stored artworkName; compute refundableAmount=artworkPrice*0.10; reset rewardPoints to reflect per-purchase model; increment cancelCount, decrement buyCount,

	clear purchase/pricing fields, set isBought=false; return message including refund.
--	---

Table 40: Method Descriptions 40

Method Signature	private void terminateVisitor()
Method Description	Hard resets a Standard account after policy violation to prevent further misuse.
Parameters	—
Return Type	void
Logic	Set isActive=false and isEligibleForDiscountUpgrade=false; zero visitCount, cancelCount, buyCount, rewardPoints; clear artworkName, artworkPrice, discountAmount, finalPrice, refundableAmount.

Table 41: Method Descriptions 41

Method Signature	@Override public void generateBill()
Method Description	Produces a stable console invoice for the current Standard purchase that mirrors earlier UI numbers.
Parameters	—
Return Type	void
Logic	If buyCount>0, print identity, artwork, original price, discountAmount, finalPrice; otherwise print an informative message.

Table 42: Method Descriptions 42

Method Signature	@Override public void display()
Method Description	Extends base diagnostics with Standard policy state for quick verification.
Parameters	—
Return Type	void
Logic	Calls <code>super.display()</code> then prints <code>isEligibleForDiscountUpgrade</code> , <code>visitLimit</code> , and <code>discountPercent</code> to confirm upgrade paths.

Table 43: Method Descriptions 43

6.3. EliteVisitor Methods

Method Signature	public EliteVisitor(int visitorId, String fullName, String gender, String contactNumber, String registrationDate, double ticketCost, String ticketType)
Method Description	Initializes an Elite account with premium benefits disabled until the user earns them via rewards.
Parameters	(same as base)
Return Type	None
Logic	Calls <code>super(...)</code> . Sets <code>assignedPersonalArtAdvisor=false</code> and <code>exclusiveEventAccess=false</code> so perks are granted only when thresholds are met.

Table 44: Method Descriptions 44

Method Signature	public boolean getIsAssignedPersonalArtAdvisor()
Method Description	Indicates whether a personal art advisor has been granted based on rewards.
Parameters	—
Return Type	boolean
Logic	Returns assignedPersonalArtAdvisor exactly as last set by assignPersonalArtAdvisor().

Table 45: Method Descriptions 45

Method Signature	public boolean getIsExclusiveEventAccess()
Method Description	Indicates whether the visitor currently has access to exclusive events, usually dependent on advisor assignment.
Parameters	—
Return Type	boolean
Logic	Returns exclusiveEventAccess which is toggled by checkExclusiveAccess().

Table 46: Method Descriptions 46

Method Signature	public boolean assignPersonalArtAdvisor()
Method Description	Awards a personal advisor when rewardPoints exceed the configured threshold, enabling premium guidance.
Parameters	—
Return Type	boolean

Logic	If <code>rewardPoints > 5000</code> , set <code>assignedPersonalArtAdvisor=true</code> ; return the resulting state. Idempotent on repeated calls.
-------	---

Table 47: Method Descriptions 47

Method Signature	<code>public boolean checkExclusiveAccess()</code>
Method Description	Enables exclusive event access once prerequisite benefits are active (advisor assigned).
Parameters	—
Return Type	<code>boolean</code>
Logic	If <code>assignedPersonalArtAdvisor</code> is true, set <code>exclusiveEventAccess=true</code> and return true; otherwise return false.

Table 48: Method Descriptions 48

Method Signature	<code>@Override public String buyProduct(int visitorID, String artworkName, double artworkPrice)</code>
Method Description	Performs Elite purchase validations (identity, activity, duplication) and records the transaction.
Parameters	<code>visitorID</code> , <code>artworkName</code> , <code>artworkPrice</code>
Return Type	<code>String</code>
Logic	Require <code>isActive</code> and correct <code>visitorID</code> ; prevent duplicates; store details; set <code>isBought=true</code> ; <code>buyCount++</code> ; return confirmation.

Table 49: Method Descriptions 49

Method Signature	@Override public double calculateDiscount()
Method Description	Applies flat Elite discount (40%) and updates finalPrice for ensuing rewards and billing.
Parameters	—
Return Type	double
Logic	If !isBought return 0; set discountAmount=artworkPrice*0.40; finalPrice=artworkPrice-discountAmount; return discountAmount.

Table 50: Method Descriptions 50

Method Signature	@Override public double calculateRewardPoint()
Method Description	Computes Elite reward points using a larger multiplier to reflect premium membership.
Parameters	—
Return Type	double
Logic	If !isBought return 0; if finalPrice not ready, call calculateDiscount(); set rewardPoints = finalPrice * 10; return rewardPoints.

Table 51: Method Descriptions 51

Method Signature	<code>@Override public String cancelProduct(int visitorID, String artworkName, String cancellationReason)</code>
Method Description	Reverses an Elite purchase under policy, granting a high refund rate and resetting financial/benefit state.
Parameters	visitorID, artworkName, cancellationReason
Return Type	String
Logic	Validate visitorID/name and cancellation cap; set refundableAmount=artworkPrice*0.95; reset rewardPoints; increment cancelCount, decrement buyCount; clear artwork/pricing fields; set isBought=false; return message including refund.

Table 52: Method Descriptions 52

Method Signature	<code>private void terminateVisitor()</code>
Method Description	Fully resets Elite account after cancellation policy is exceeded, removing perks and deactivating activity.
Parameters	—
Return Type	void
Logic	Set assignedPersonalArtAdvisor=false; exclusiveEventAccess=false; isActive=false; zero visitCount, cancelCount, buyCount, rewardPoints; clear artworkName, artworkPrice, discountAmount, finalPrice, refundableAmount.

Table 53: Method Descriptions 53

Method Signature	@Override public void generateBill()
Method Description	Prints an Elite invoice that reflects the 40% discount and final payable amount consistently with prior UI displays.
Parameters	—
Return Type	void
Logic	If buyCount>0, print identity, artwork, original price, discountAmount, finalPrice; else print a friendly message.

Table 54: Method Descriptions 54

Method Signature	@Override public void display()
Method Description	Extends base diagnostics with Elite-specific perks to verify the reward-to-benefit pipeline.
Parameters	—
Return Type	void
Logic	Calls super.display() then prints assignedPersonalArtAdvisor and exclusiveEventAccess.

Table 55: Method Descriptions 55

7. Testing

7.1. Test 1a: Verify Program Compilation

Objective	To ensure that the program compiles successfully without any syntax or dependency errors, confirming that all Java files are correctly written and contain no semantic/syntax mistakes.
Action	The file ArtGalleryGUI.java was compiled using <code>`javac ArtGalleryGUI.java`</code> in the terminal/command prompt, verifying that the source code compiles.
Expected Output	The code should compile cleanly, generating .class files for all the Java classes. No compilation failures or errors should appear; otherwise the program is ready for execution.
Actual Output	The code compiled without any syntax or dependency errors, all .class files were generated successfully in the directory.
Result	Pass

Table 56: Test 1a

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Adhish\Coursework-Adhish-SEM2> javac *.java
PS C:\Users\Adhish\Coursework-Adhish-SEM2> |
```

Figure 13: Test 1a

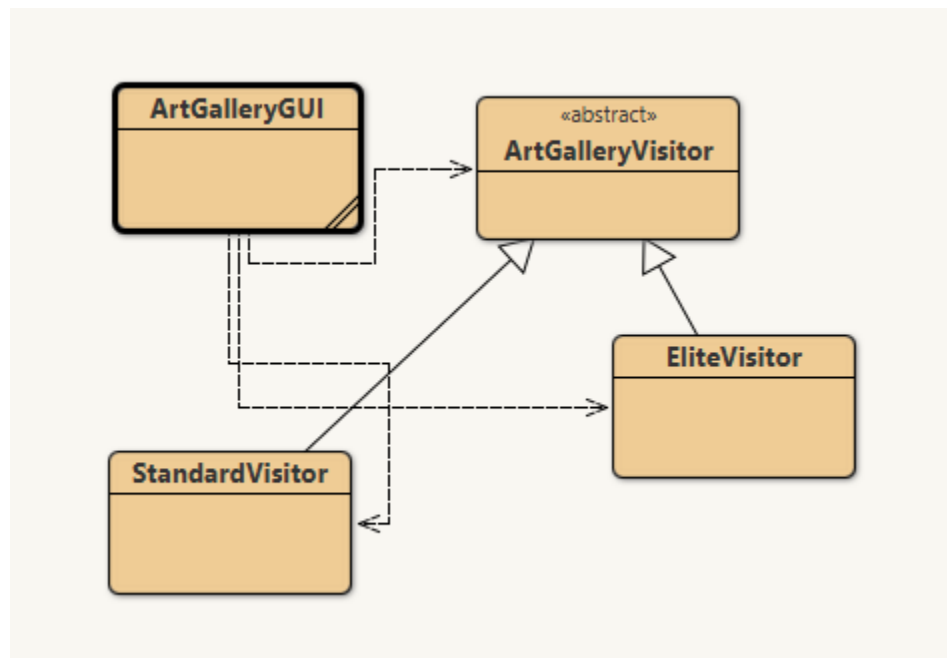


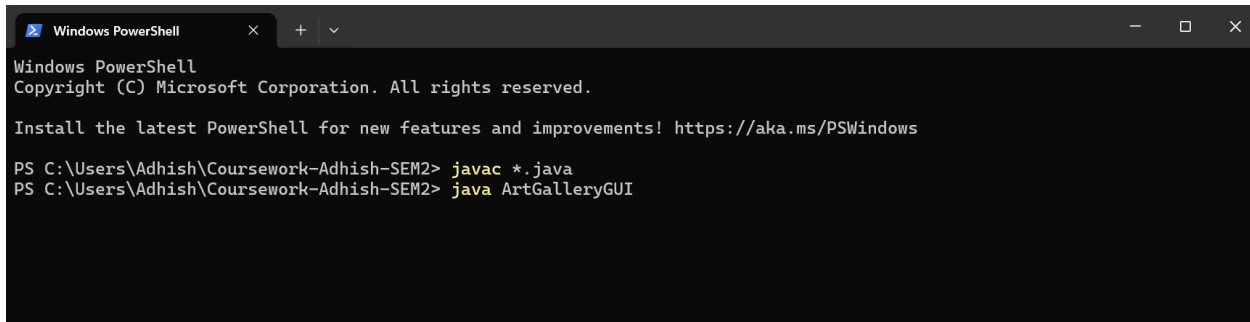
Figure 14: Test 1a

7.2. Test 1b: Verify GUI Launch

Objective	To verify that the graphical user interface (GUI) launches properly and that all panels (Dashboard, Login, Visitors, and Artworks) are accessible from the navigation bar.
Action	The program was launched using `java ArtGalleryGUI`. The GUI window was

	expected to appear with full layout and interactive components.
Expected Output	The GUI should launch successfully, showing the navigation bar with different views. The program should visually display Dashboard, Login, Visitors, and Artworks panels without any runtime errors.
Actual Output	The GUI launched correctly, showing all four expected panels. Navigation functions worked as expected, and no exceptions were raised.
Result	Pass

Table 57: Test 1b



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Adhish\Coursework-Adhish-SEM2> javac *.java
PS C:\Users\Adhish\Coursework-Adhish-SEM2> java ArtGalleryGUI
```

Figure 15: Test 1b

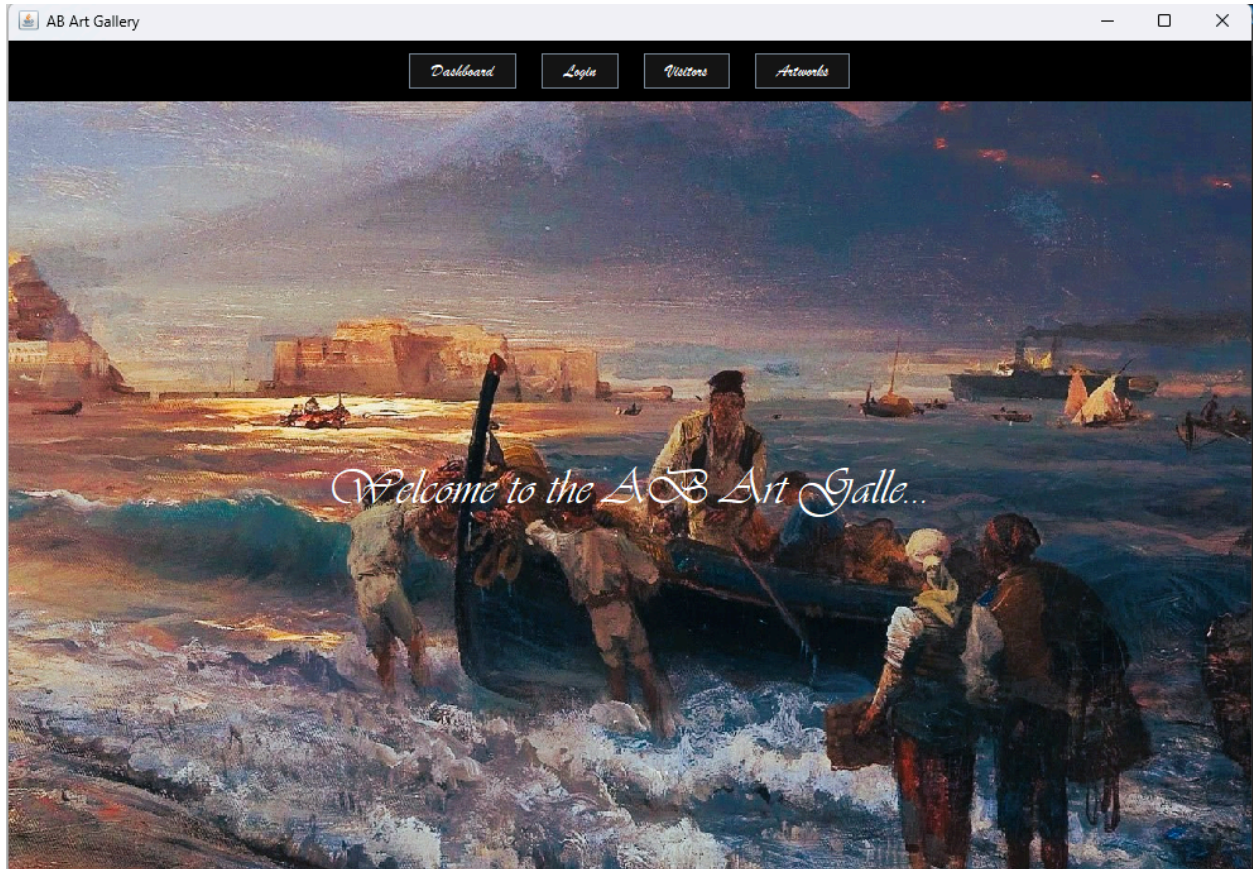


Figure 16: Test 1b

7.3. Test 2a: Add Standard Visitor

Objective	To confirm that a Standard visitor can be registered in the system with valid details such as Visitor ID, Name, Gender, Contact Number, Registration Date, and Ticket Type.
Action	In the Login panel, the Visitor ID, full name, gender, contact number, and date fields plus 'Standard' were entered. The Add Visitor button was clicked to register the visitor.
Expected Output	The system should validate the inputs, add the Standard visitor to the Visitors table, and display the visitor's details with a confirmation dialog.

Actual Output	The Standard visitor was successfully registered. The details were added to the table, and a confirmation popup notified successful registration.
Result	Pass

Table 58: Test 2a

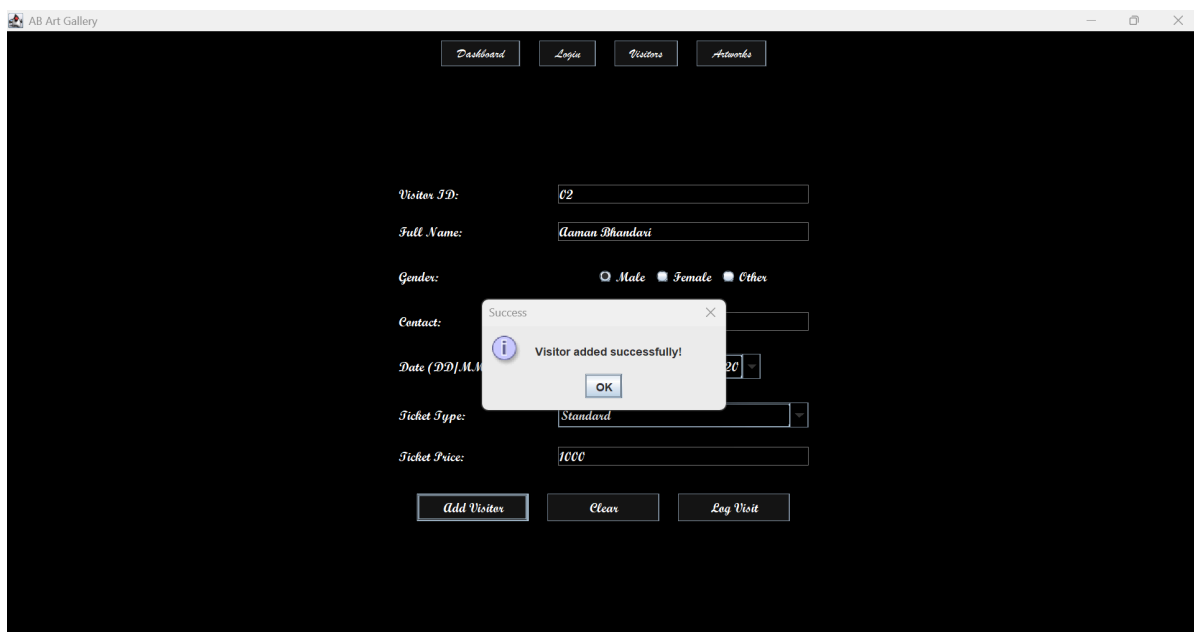


Figure 17: Test 2a

CS4001NI Programming

The screenshot shows the 'Visitors' tab of the AB Art Gallery application. The form contains the following fields and values:

- Visitor ID: 02
- Full Name: Aaman Bhandari
- Gender: ☒ Male ☐ Female ☐ Other
- Contact: (empty)
- Date (D): (empty)
- Ticket Type: Standard
- Ticket Price: 1000

Buttons at the bottom: Add Visitor, Clear, Log Visit.

A modal dialog titled 'Duplicate Visitor' is displayed in the center, showing a warning icon and the message: 'Visitor ID already exists. Duplicate not allowed.' with an 'OK' button.

Figure 18: Test 2a

The screenshot shows the 'Visitors' tab of the AB Art Gallery application. It displays two tables: 'Standard Visitors' and 'Elite Visitors'.

Standard Visitors

ID	Name	Gender	Contact	Date	Type	Price	Visits	Upgrade?	Discount ..
1	Adhish ..	Male	0000000...	11Jan20..	Standard	1000.0	1	No	10%
2	Aaman ..	Male	0000000...	5Feb20..	Standard	1000.0	0	No	10%

Elite Visitors

ID	Name	Gender	Contact	Date	Type	Price	Visits	Advisors	Event Acc..
3	Anuj Bh...	Male	0000000...	5Feb20..	Elite	2000.0	0	No	No

Buttons at the bottom: Visitor ID: (input), Assign Per..., Check Upg..., Clear, Read File

Figure 19: Test 2a

7.4. Test 2b: Add Elite Visitor

Objective	To confirm that an Elite visitor can be added with valid details and is properly categorized under Elite visitors.
Action	In the Login panel, Visitor ID, name, gender, contact number, and date were entered. The Ticket Type 'Elite' was selected and Add Visitor clicked.
Expected Output	The Elite visitor should be added to the Visitors table, showing the Elite category, and a confirmation dialog should appear.
Actual Output	The Elite visitor was successfully added and displayed correctly in the Visitors table under Elite category. Confirmation dialog worked as expected.
Result	Pass

Table 59: Test 2b

The screenshot shows the 'AB Art Gallery' application window. At the top, there are four buttons: 'Dashboard', 'Login', 'Visitors', and 'Artworks'. The 'Visitors' button is highlighted. Below the buttons, there is a form with the following fields: 'Visitor ID:' with the value 'C3', 'Full Name:' with the value 'Anuj Bhandari', 'Gender:' with radio buttons for 'Male' (selected), 'Female', and 'Other', 'Contact:' (empty), 'Date (DD/MM)' (empty), 'Ticket Type:' with a dropdown menu showing 'Elite', and 'Ticket Price:' with the value '2000'. At the bottom of the form, there are three buttons: 'Add Visitor', 'Clear', and 'Log Visit'. A modal dialog box titled 'Success' is displayed in the center, showing a green checkmark icon and the message 'Visitor added successfully!' with an 'OK' button.

Figure 20: Test 2b

The screenshot shows the 'AB Art Gallery' application window. At the top, there are four buttons: 'Dashboard', 'Login', 'Visitors', and 'Artworks'. The 'Visitors' button is highlighted. Below the buttons, there is a form with the following fields: 'Visitor ID:' with the value 'C3', 'Full Name:' with the value 'Anuj Bhandari', 'Gender:' with radio buttons for 'Male' (selected), 'Female', and 'Other', 'Contact:' (empty), 'Date (DD/MM)' (empty), 'Ticket Type:' with a dropdown menu showing 'Elite', and 'Ticket Price:' with the value '2000'. At the bottom of the form, there are three buttons: 'Add Visitor', 'Clear', and 'Log Visit'. A modal dialog box titled 'Duplicate Visitor' is displayed in the center, showing a yellow warning triangle icon and the message 'Visitor ID already exists. Duplicate not allowed.' with an 'OK' button.

Figure 21: Test 2b

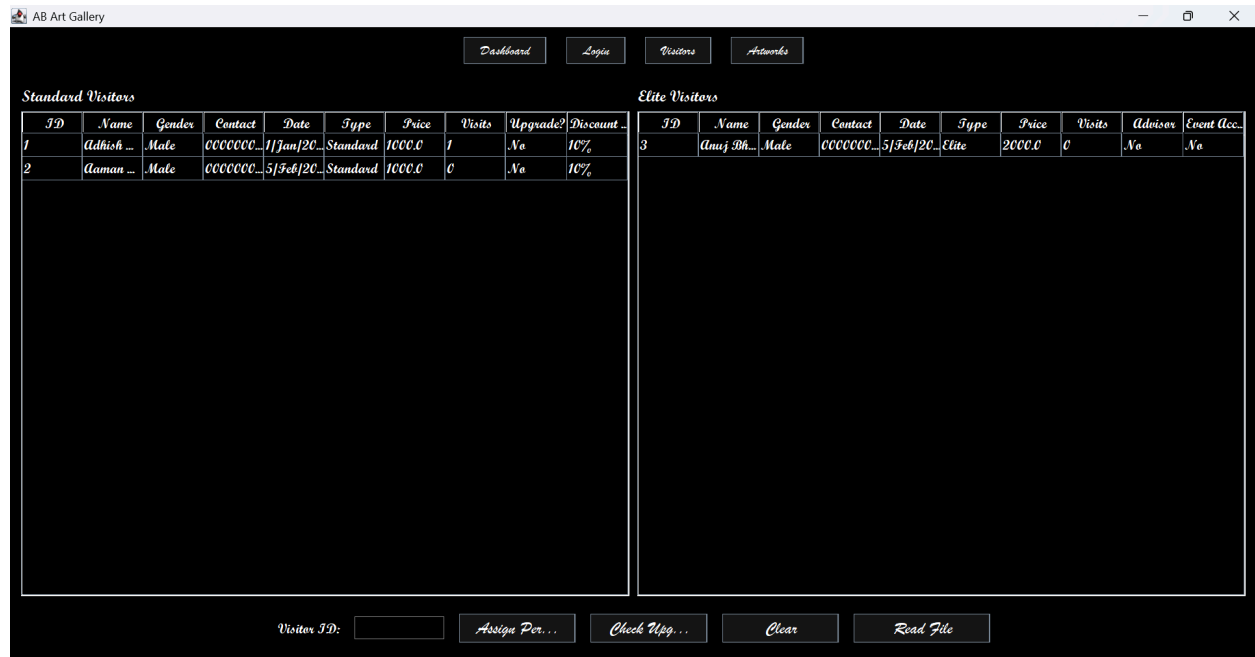


Figure 22: Test 2b

7.5. Test 3a: Log Visit

Objective	To check that the system logs a visit for an existing visitor and increments their visit count.
Action	A visitor ID was entered and the Log Visit button was clicked.
Expected Output	The visitor's visit count should increment by one, and the updated visit value should display in the Visitors table.
Actual Output	The visit was logged successfully, with the visit count increasing and updating in the Visitors table.
Result	Pass

Table 60: Test 3a

AB Art Gallery

Dashboard Login Visitors Artworks

Visitor ID: 02

Full Name: Aaman Bhandari

Gender: ☒ Male ☐ Female ☐ Other

Contact:

Date (DD/MM): 20

Ticket Type: Standard

Ticket Price: 1000

Add Visitor Clear Log Visit

Visit Logged

Visit logged for visitor ID 2
Total visits: 2

OK

Figure 23: Test 3a

AB Art Gallery

Dashboard Login Visitors Artworks

Standard Visitors

ID	Name	Gender	Contact	Date	Type	Price	Visits	Upgrade?	Discount
1	Adhish ...	Male	0000000...	11-Jan-20...	Standard	1000.0	1	No	10%
2	Aaman ...	Male	0000000...	5-Feb-20...	Standard	1000.0	2	No	10%

Elite Visitors

ID	Name	Gender	Contact	Date	Type	Price	Visits	Advisor	Event Acc..
3	Anuj Bh...	Male	0000000...	5-Feb-20...	Elite	2000.0	0	No	No

Visitor ID:

Assign Per... Check Upg... Clear Read File

Figure 24: Test 3a

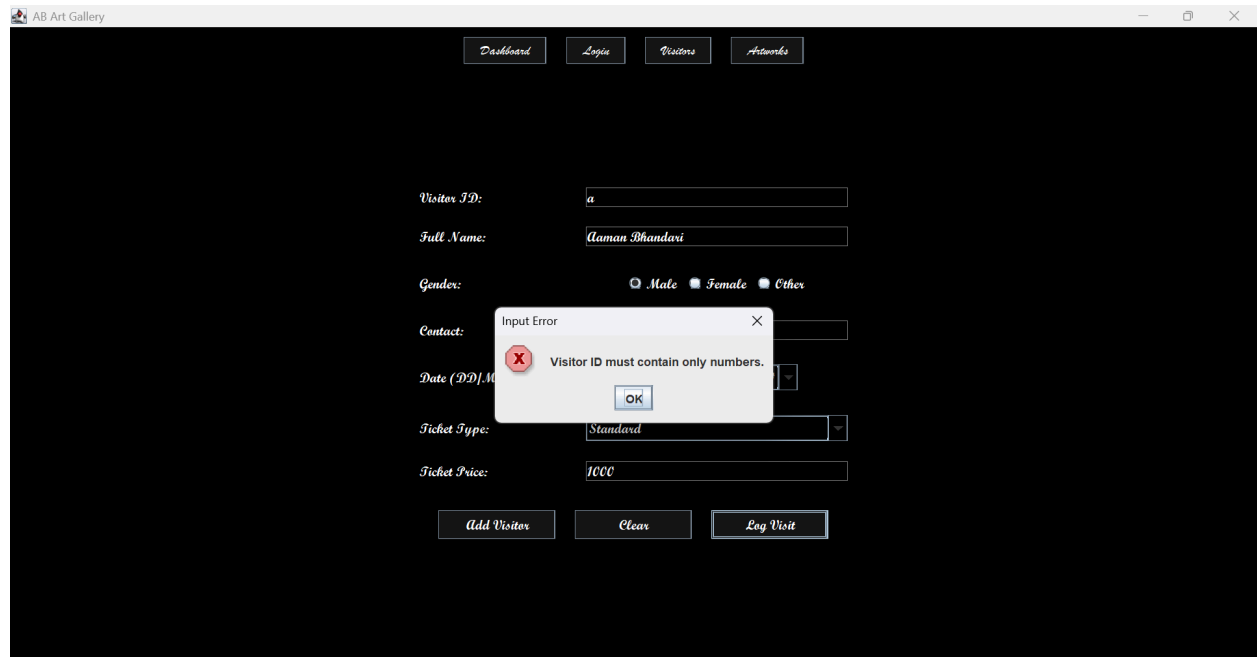


Figure 25: Test 3a

7.6. Test 3b: Buy Product

Objective	To test that an artwork can be purchased by a registered visitor.
Action	The visitor ID, artwork name, and price were entered and the Buy button clicked.
Expected Output	The system should register the purchase, deduct applicable discounts, and display the artwork in the purchase table.
Actual Output	The artwork purchase was successful, with discounted price applied and updated purchase details shown in the table.
Result	Pass

Table 61: Test 3b

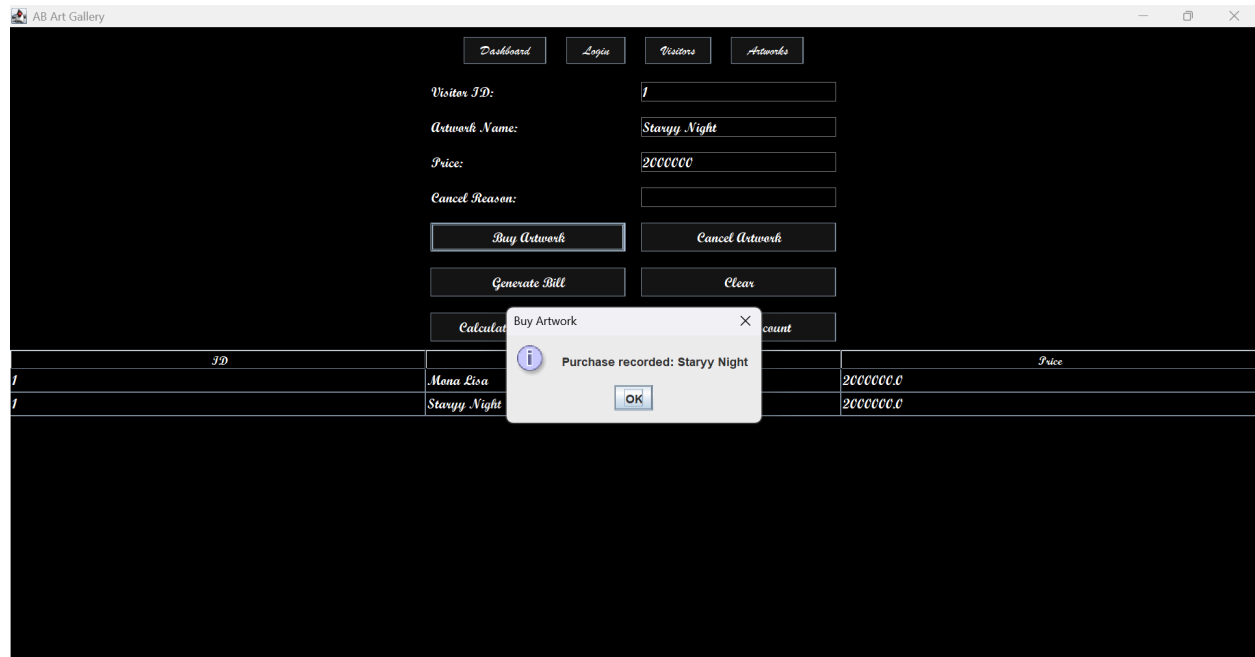


Figure 26: Test 3b

7.7. Test 3c: Cancel Product

Objective	To verify cancellation functionality including refund calculation.
Action	A visitor ID, artwork details, and reason for cancellation were provided, then Cancel button was clicked.
Expected Output	The artwork should be removed from purchase list, cancellation count incremented, and refund displayed.
Actual Output	The cancellation was processed successfully. Refund amount updated and cancellation count incremented in table.
Result	Pass

Table 62: Test 3c

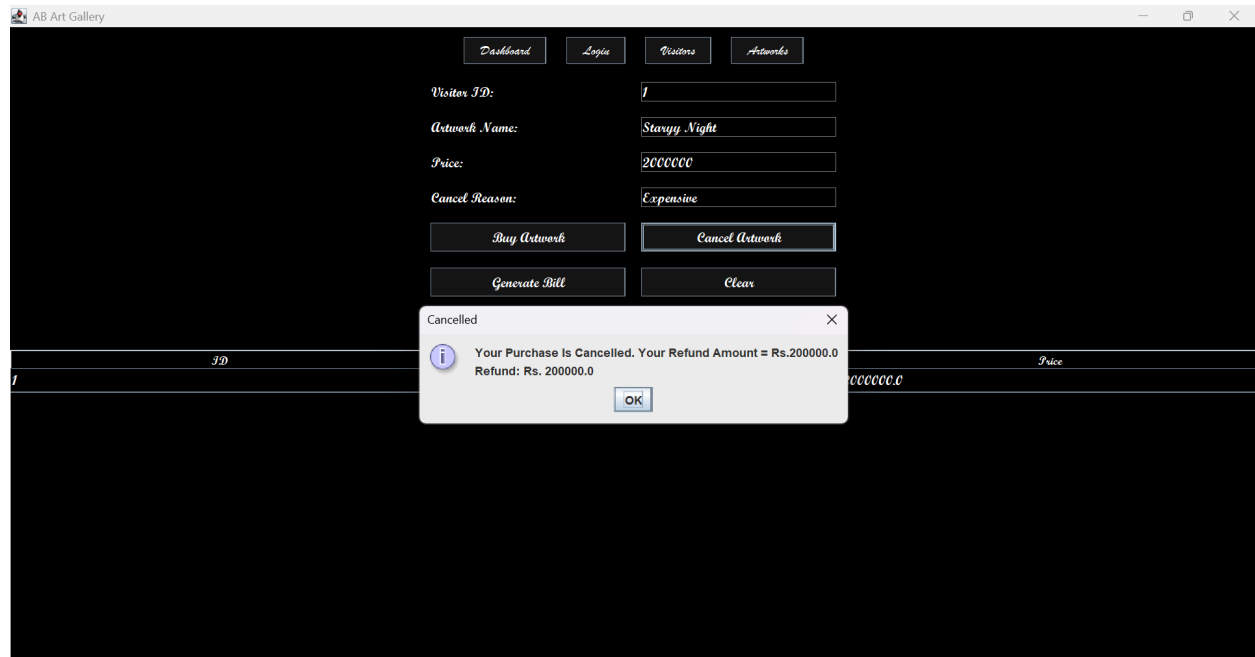


Figure 27: Test 3c

7.8. Test 3d: Assign Personal Advisor (Elite)

Objective	To verify that an Elite visitor with sufficient reward points gets a personal art advisor assigned.
Action	An Elite visitor was logged multiple times, purchases made to accumulate reward points until threshold reached.
Expected Output	System should assign a personal advisor once reward points exceed the threshold and reflect this in Elite visitor's record.
Actual Output	The Elite visitor crossed the reward points threshold and was successfully assigned a personal advisor.
Result	Pass

Table 63: Test 3d

CS4001NI Programming

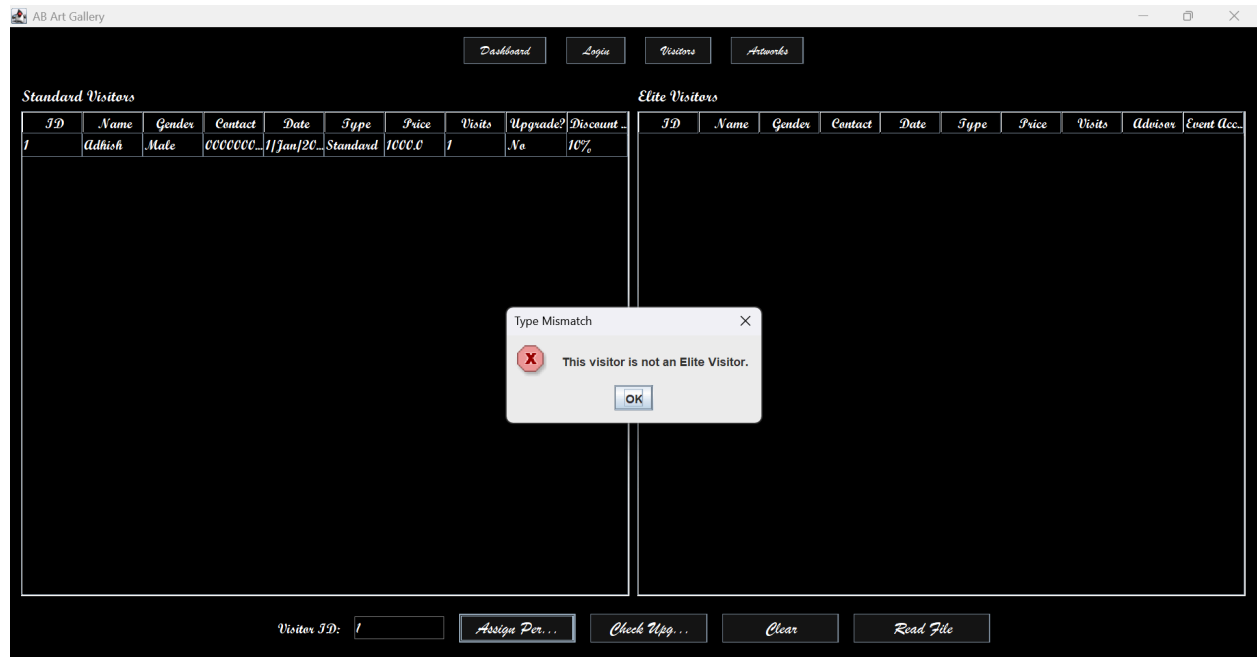


Figure 28: Test 3d

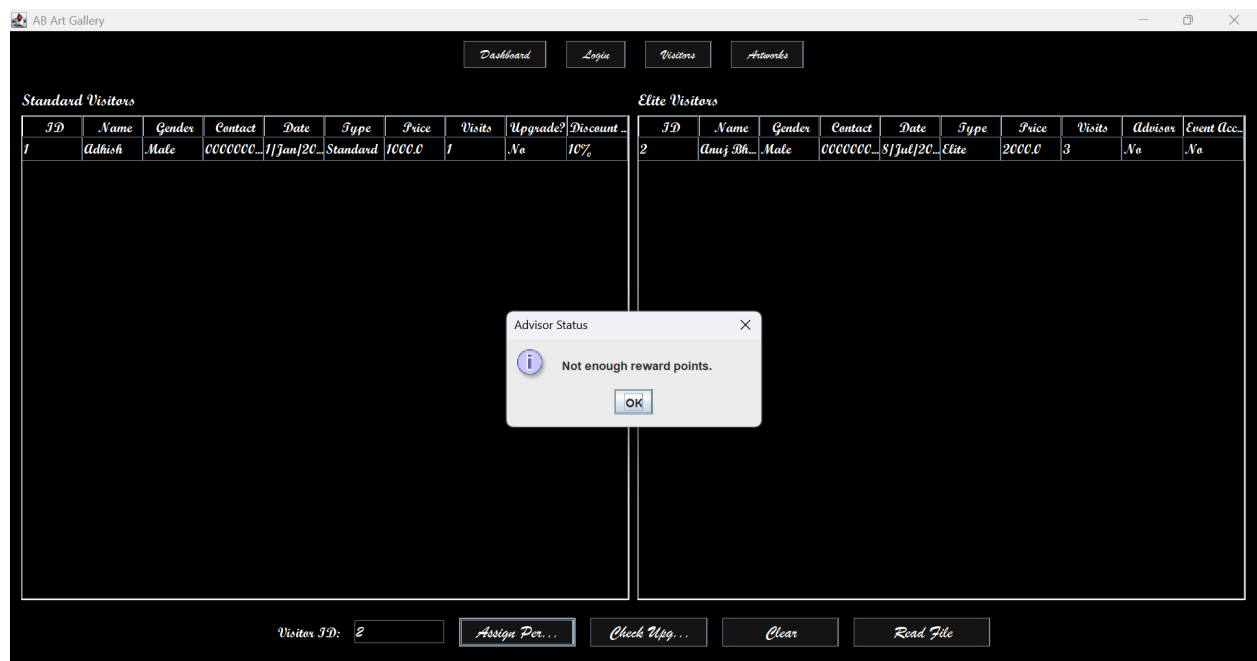


Figure 29: Test 3d

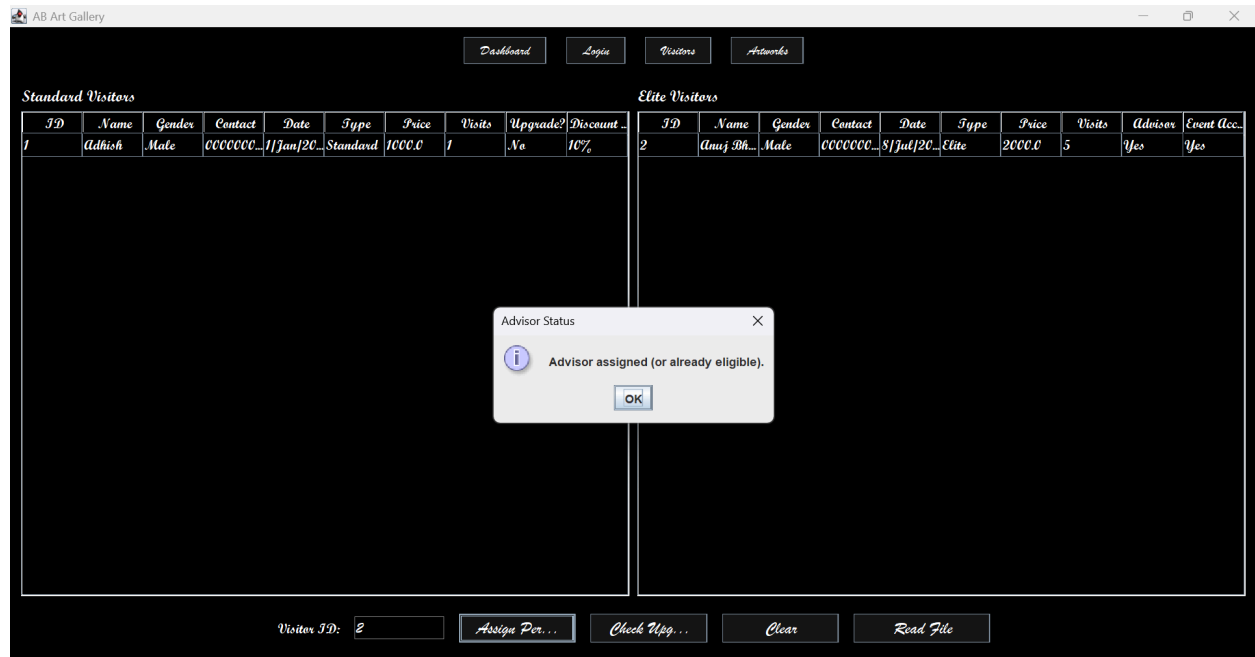


Figure 30: Test 3d

7.9. Test 4a: Check Discount Upgrade

Objective	To confirm that Standard visitors get upgraded discount after reaching the visit limit.
Action	A Standard visitor was logged repeatedly until visit count $\geq$ limit.
Expected Output	The system should upgrade discount percentage for Standard visitor (e.g., from 10% to 20%).
Actual Output	Discount upgrade triggered successfully after visit limit, updating discount for Standard visitor.
Result	Pass

Table 64: Test 4a

CS4001NI Programming

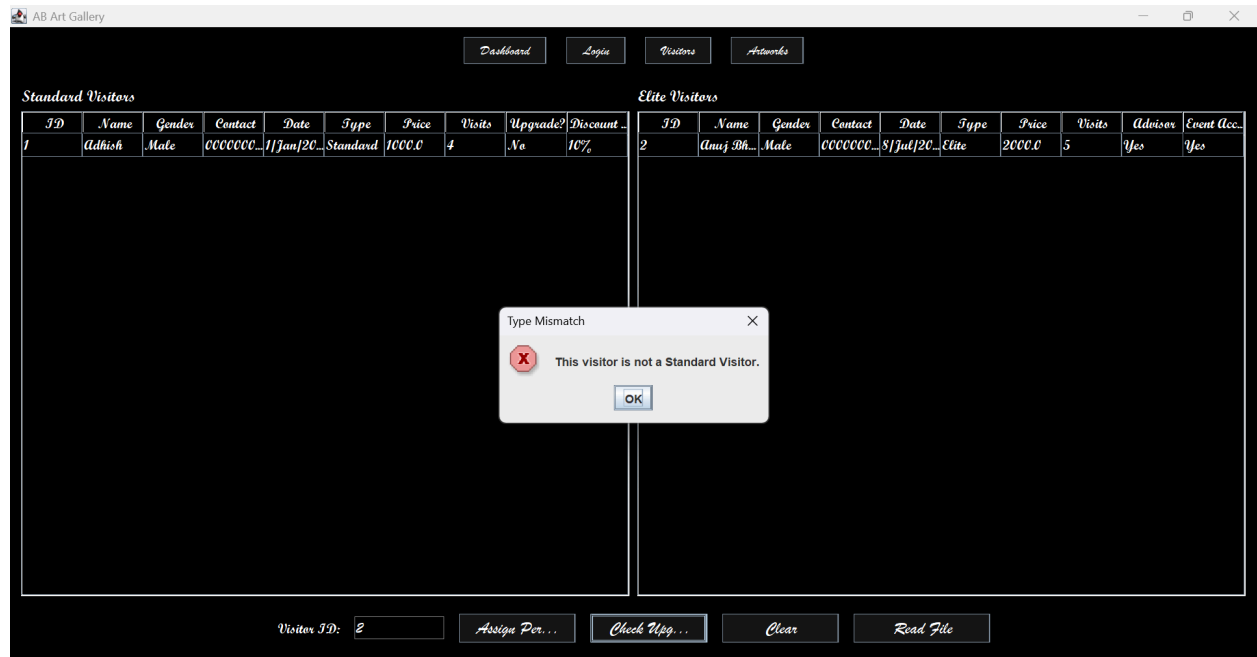


Figure 31: Test 4a

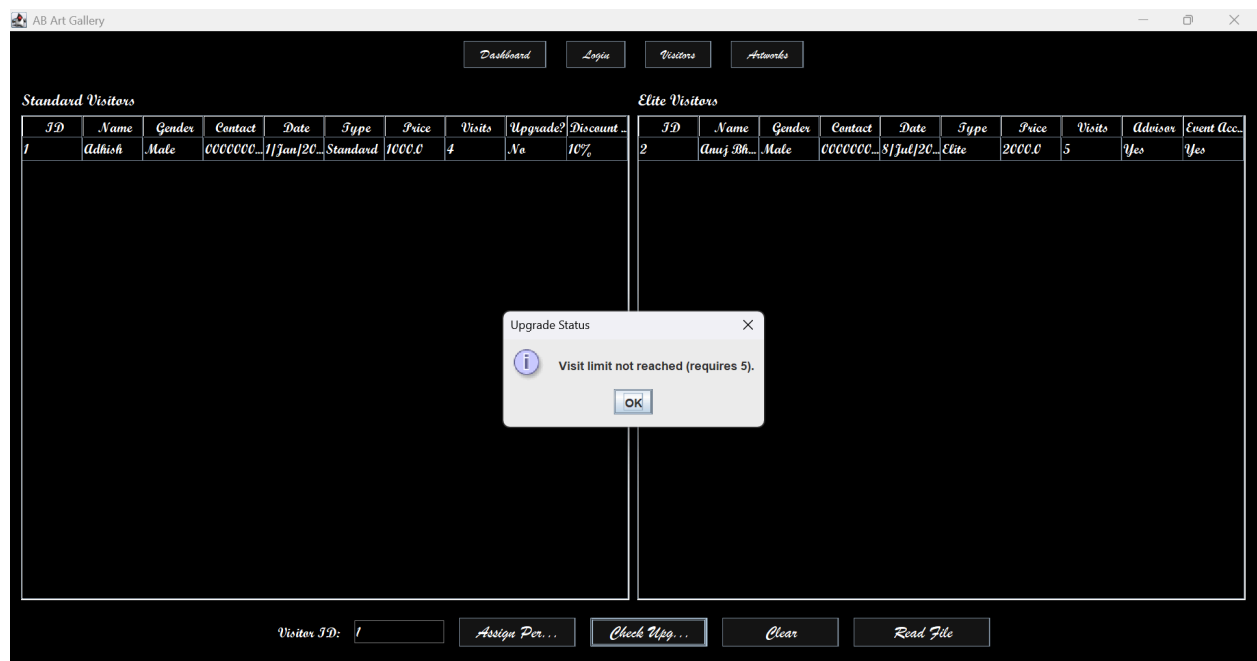


Figure 32: Test 4a

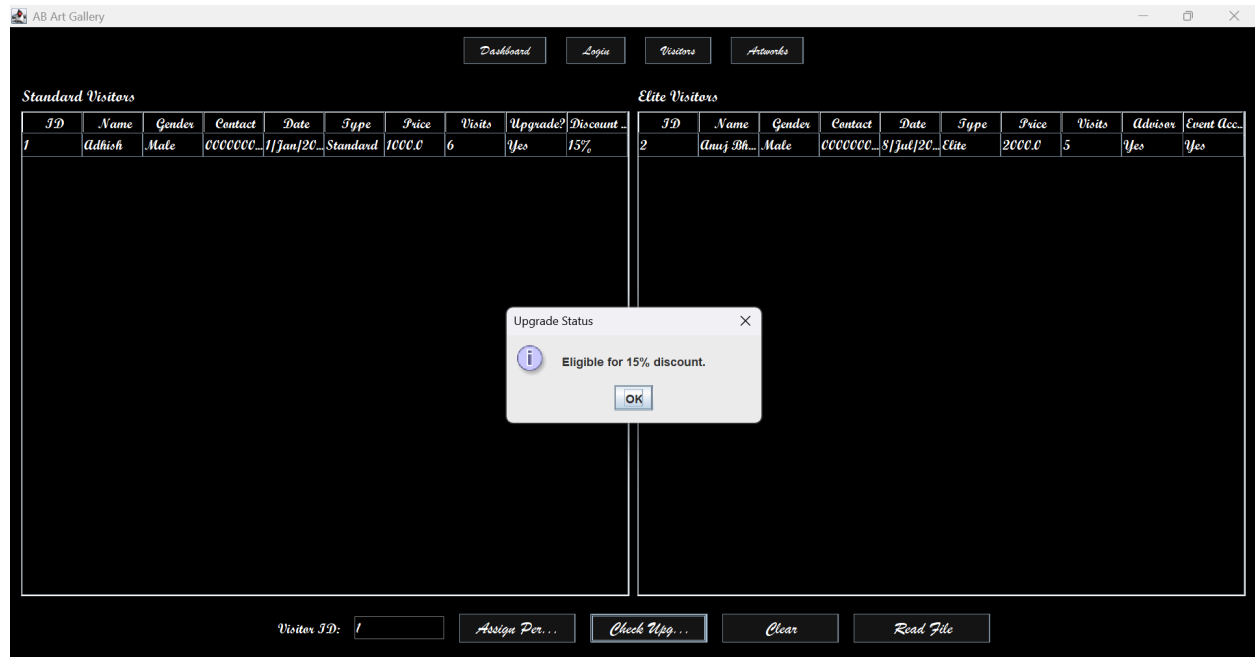


Figure 33: Test 4a

7.10. Test 4b: Calculate Discount

Objective	To ensure discounts are calculated correctly during purchase.
Action	Standard and Elite visitors purchased artworks at various prices.
Expected Output	The purchase prices should reflect correct discount percentages (e.g., 15% for Standard after upgrade, 40% for Elite).
Actual Output	Discounts applied correctly to purchase prices as per visitor type and status.
Result	Pass

Table 65: Test 4b

CS4001NI Programming

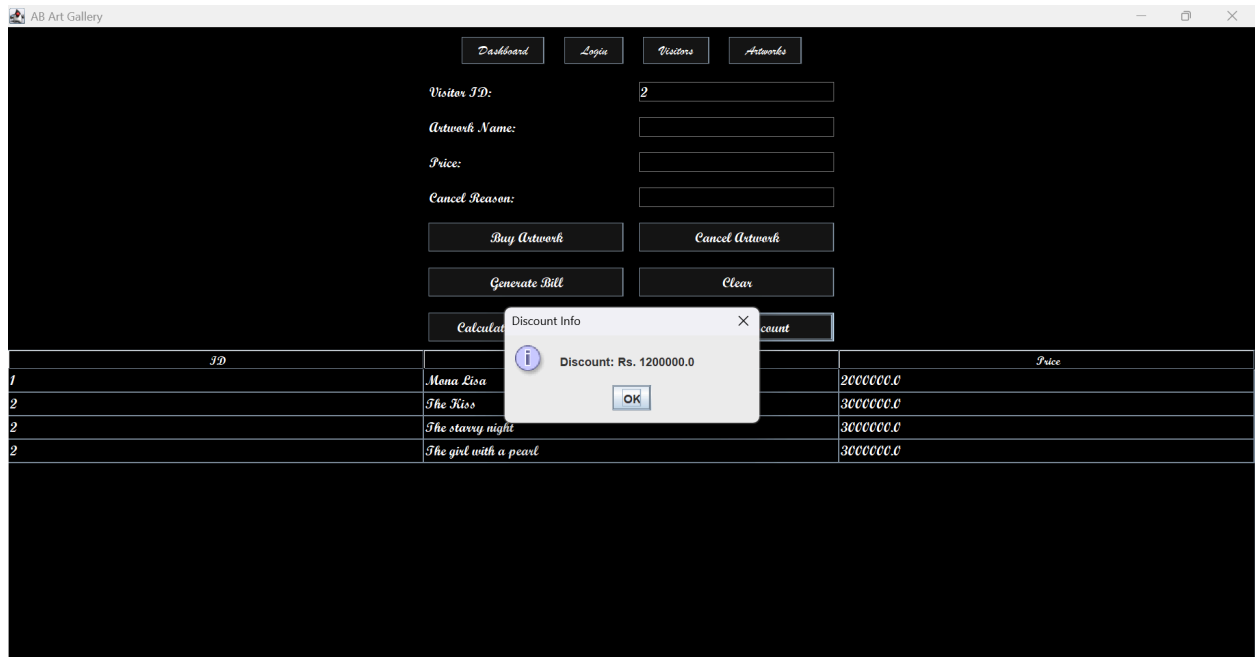


Figure 34: Test 4b

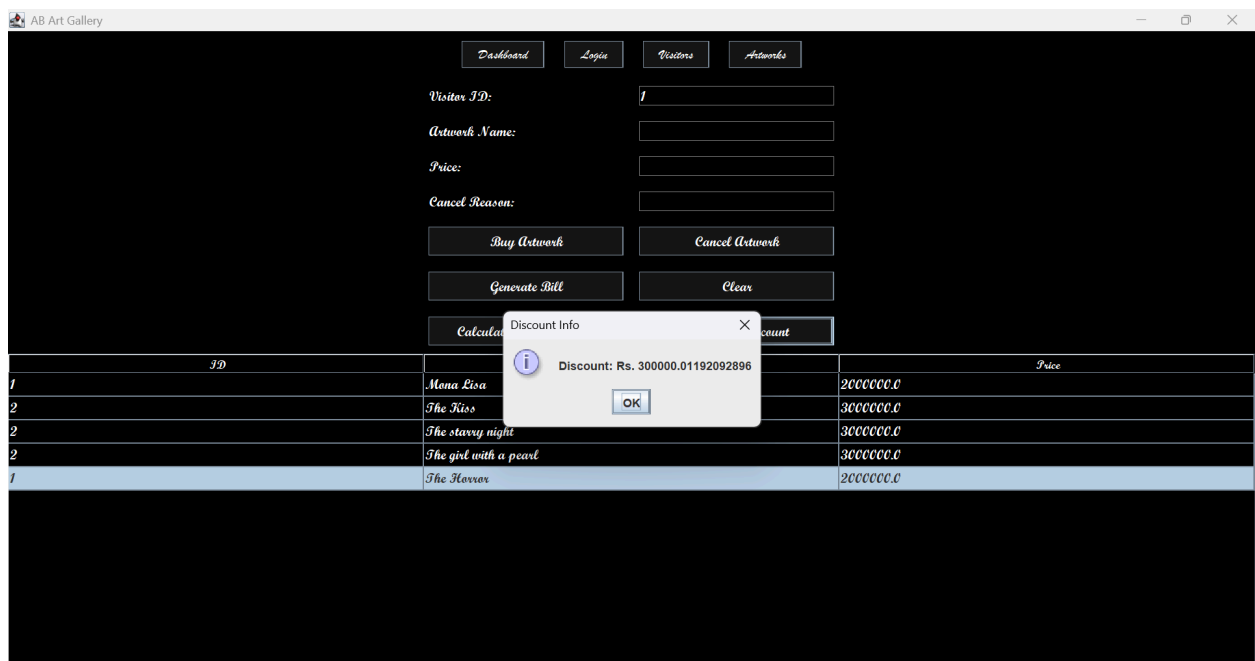


Figure 35: Test 4b

7.11. Test 4c: Calculate Reward Points

Objective	To verify reward points accumulate per purchase.
Action	Visitors purchased multiple artworks.
Expected Output	Reward points should increment proportionally to each purchase based on visitor type rules.
Actual Output	Reward points were calculated and accumulated correctly in visitor records.
Result	Pass

Table 66: Test 4c

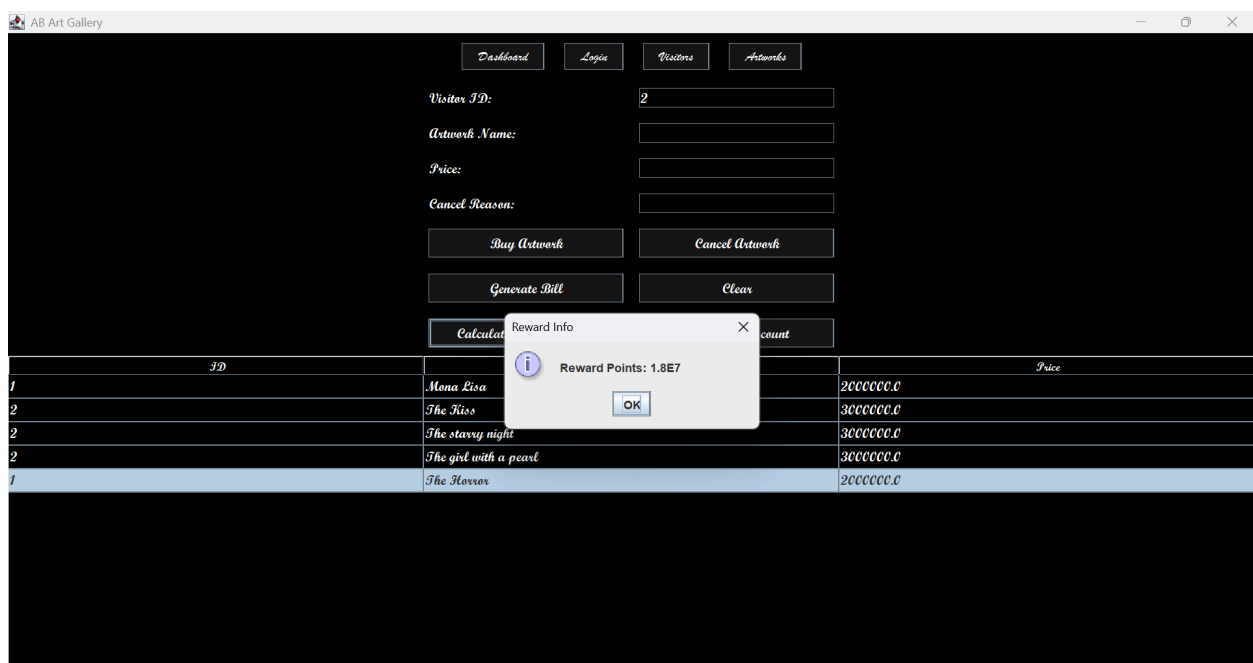


Figure 36: Test 4c

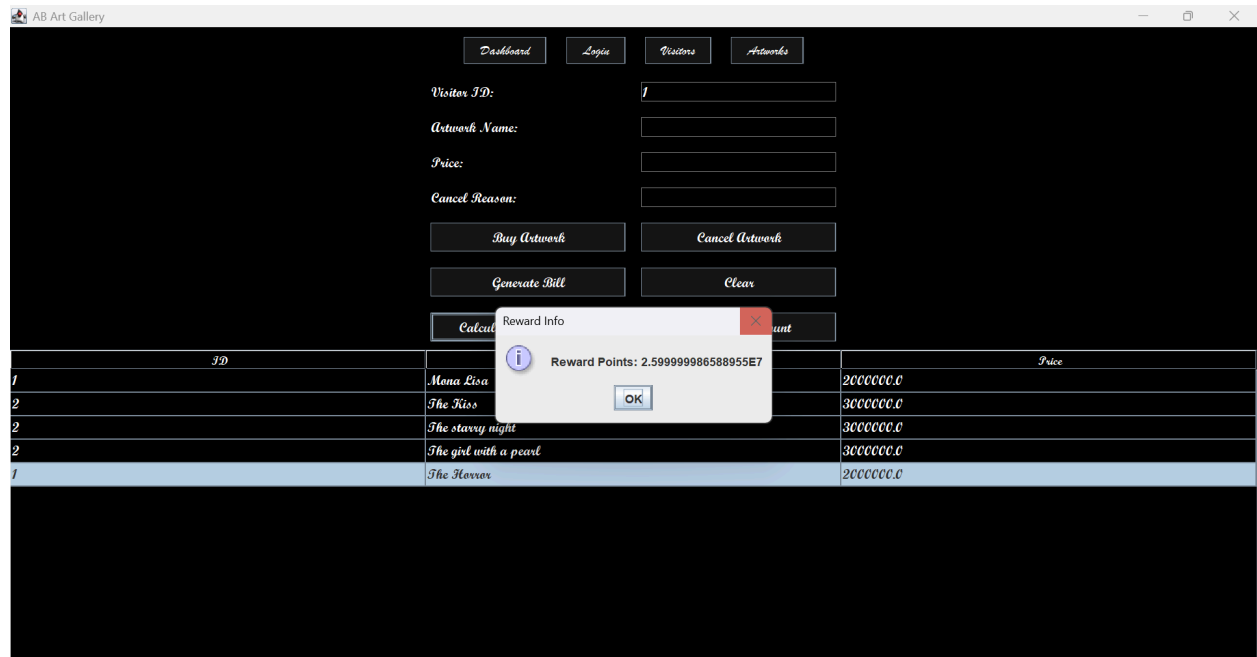


Figure 37: Test 4c

7.12. Test 5a: Save to File

Objective	To check that visitor data can be stored in a file.
Action	Visitors and purchases were added, then Save button clicked.
Expected Output	All visitor details should be written into the file including ID, name, ticket type, and purchases.
Actual Output	Visitor data was successfully stored in the file as expected.
Result	Pass

Table 67: Test 5a

```

==== Standard Visitors ====
ID   Name       Gender  Contact   RegDate   Ticket   Cost    Visits  Discount  UpgradeEligible
-----
1    Adhish Bhandari  Male    0000000001 1/Jan/2020 Standard 1000.00 6       15%      Yes

==== Elite Visitors ====
ID   Name       Gender  Contact   RegDate   Ticket   Cost    Visits  ArtAdvisor  EventAccess
-----
2    Aaman Bhandari  Male    0000000002 5/Dec/2023 Elite    2000.00 5       Yes      Yes
  
```

Figure 38: Test 5a

7.13. Test 5b: Read from File

Objective	To check that visitor data can be reloaded from a file into GUI tables.
Action	The Read button was clicked after saving file.
Expected Output	Visitor data should be reloaded from the file and displayed correctly in Visitors table with full details.
Actual Output	Visitor data was successfully reloaded from the file and populated into the Visitors table correctly.
Result	Pass

Table 68: Test 5b

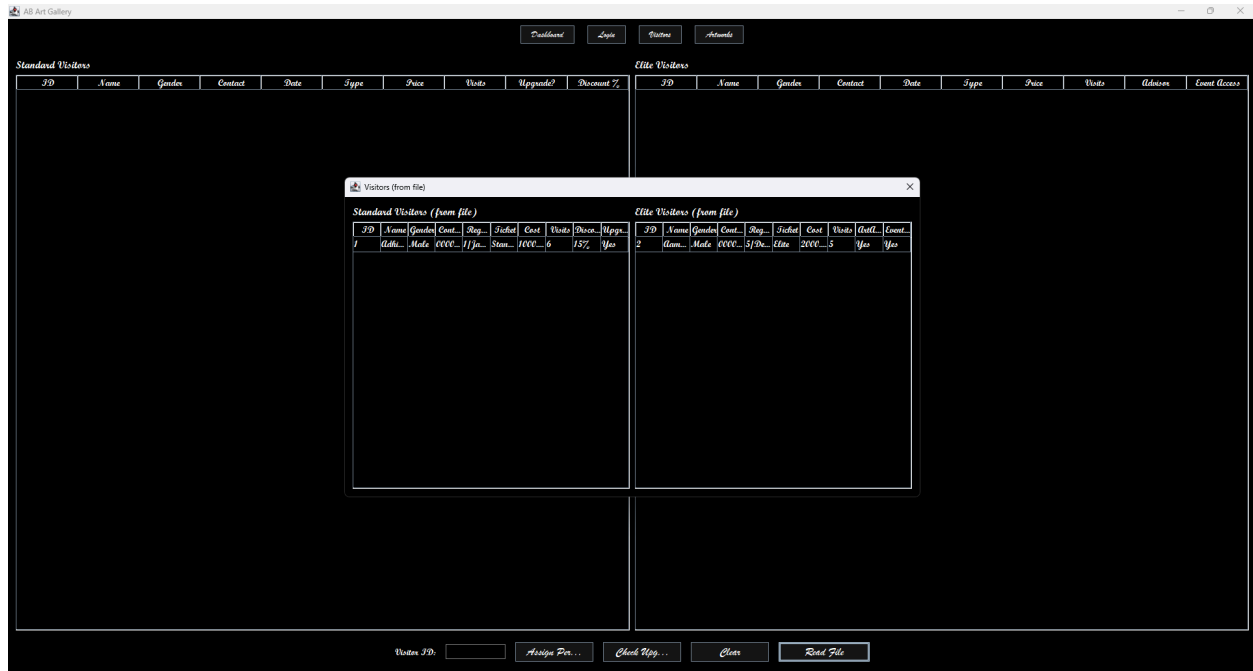


Figure 39: Test 5b

8. Error Detection and Correction

8.1. Syntax Error

Field	Content
Error Title	Unresolved classes/constructors when reading visitors.txt
Error Type	Syntax / API usage (compile-time)
Evidence	IDE shows red underlines and compile errors on BufferedReader, InputStreamReader, FileInputStream, StandardCharsets.
Root Cause	Missing imports or using a constructor signature not available in the configured JDK.

Affected Code	BufferedReader br = new BufferedReader(new InputStreamReader(new FileInputStream(f), java.nio.charset.StandardCharsets.UTF_8));
Fix	BufferedReader br = new java.io.BufferedReader(new java.io.FileReader(f));
Why This Works	Avoids unavailable classes/signatures and uses a compatible constructor.
Regression Test	Rebuild project and click Read File. Expected: compiles with no errors, tables display correctly.

Table 69: Syntax Error

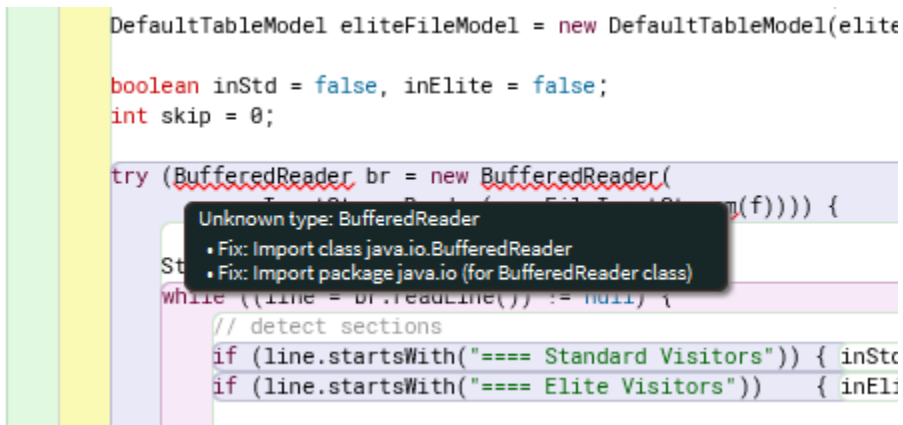


Figure 40: Syntax Error

CS4001NI Programming

```
import javax.swing.*;
import java.awt.*;
import javax.swing.table.DefaultTableModel;
import javax.swing.table.JTableHeader;
import java.awt.event.*;
import java.util.ArrayList;
import javax.swing.border.TitledBorder;
import javax.swing.JFileChooser;
import java.io.File;
import java.io.PrintWriter;
import java.io.OutputStreamWriter;
import java.io.FileOutputStream;
import java.nio.charset.StandardCharsets;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.FileInputStream;
import java.nio.charset.StandardCharsets;
```

Figure 41: Syntax Error

```
DefaultTableModel stdTableModel = new DefaultTableModel(stdCols, 0);
DefaultTableModel eliteFileModel = new DefaultTableModel(eliteCols, 0);

boolean inStd = false, inElite = false;
int skip = 0;
```

```
try (BufferedReader br = new BufferedReader(
    new InputStreamReader(new FileInputStream(f)))) {
    String line;
    while ((line = br.readLine()) != null) {
        // detect sections
        if (line.startsWith("==== Standard Visitors")) { inStd = true; inElite = false; skip = 2; c
        if (line.startsWith("==== Elite Visitors")) { inElite = true; inStd = false; skip = 2; c
        // skip header and dashed line
```

Figure 42: Syntax Error

8.2. Runtime NPE in “Read File” Dialog

Field	Content
Error Title	NullPointerException from getWindowAncestor / wrong dialog owner
Error Type	Runtime (API misuse)
Evidence	Console shows NPE when launching Read File dialog. Dialog does not appear.
Root Cause	Passing null component to SwingUtilities.getWindowAncestor or using this incorrectly inside ActionListener.
Affected Code	JDialog dlg = new JDialog(this, "Visitors (from file)", Dialog.ModalityType.APPLICATION_MODAL);
Fix	JDialog dlg = new JDialog(ArtGalleryGUI.this, "Visitors (from file)", java.awt.Dialog.ModalityType.APPLICATION_MODAL);
Why This Works	Ensures dialog has a valid owner; prevents NPE and type mismatch.
Regression Test	Launch GUI → Visitors → Read File. Expected: modal dialog opens centered; no NPE.

Table 70: Runtime NPE in “Read File” Dialog

```
JDialog dlg = new JDialog(owner, "Visitors (from file)", Dialog.ModalityType.APPLICATION_MODAL);  
dlg.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);
```

Figure 43: Runtime NPE in “Read File” Dialog

```
JDialog dlg = (owner != null)  
    ? new JDialog(owner, "Visitors (from file)", Dialog.ModalityType.APPLICATION_MODAL)  
    : new JDialog((Frame) null, "Visitors (from file)", true); // fallback if owner is null
```

Figure 44: Runtime NPE in “Read File” Dialog

8.3. Logical Refund Calculation (Standard Cancel)

Field	Content
Error Title	Incorrect refund formula for Standard visitor cancellation
Error Type	Logical / Business-rule
Evidence	Refund amount far larger than expected (e.g., refund in billions).
Root Cause	Refund computed as $\text{artworkPrice} * (\text{artworkPrice} * 0.10)$ instead of $\text{artworkPrice} * 0.10$.
Affected Code	$\text{refundableAmount} = \text{artworkPrice} * (\text{artworkPrice} * 0.10);$
Fix	$\text{refundableAmount} = \text{artworkPrice} * 0.10;$
Why This Works	Correctly implements refund = 10% of price.
Regression Test	Standard visitor cancels Rs. 10,000 artwork → refund Rs. 1,000. Purchase cleared.

Table 71: Logical Refund Calculation (Standard Cancel)

```
@Override
public String cancelProduct(int visitorID, String artworkName, String cancellationReason) {
    if (visitorID != this.visitorId) {
        return "Visitor ID mismatch. Cancellation denied.";
    }
    if (cancelCount >= cancellimit) {
        terminateVisitor();
        return "Cancel Limit Reached. Account has been terminated.";
    }
    if (buyCount == 0 || !isBought) {
        return "No Purchase To Cancel.";
    }
    if (this.artworkName == null || !this.artworkName.equals(artworkName)) {
        return "Art Work Name Mismatch.";
    }
    refundableAmount = artworkPrice * (artworkPrice * 0.10); // 10% refund but in billions
    rewardPoints -= finalPrice * 5;
    cancelCount++;
    buyCount--;
    this.cancellationReason = cancellationReason;
    isBought = false;
    this.artworkName = null;
    artworkPrice = 0;
    discountAmount = 0;
    finalPrice = 0;
    return "Your Purchase Is Cancelled. Your Refund Amount = Rs." + refundableAmount;
}
```

Figure 45: Logical Refund Calculation (Standard Cancel)

```
@Override
public String cancelProduct(int visitorID, String artworkName, String cancellationReason) {
    if (visitorID != this.visitorId) {
        return "Visitor ID mismatch. Cancellation denied.";
    }
    if (cancelCount >= cancellimit) {
        terminateVisitor();
        return "Cancel Limit Reached. Account has been terminated.";
    }
    if (buyCount == 0 || !isBought) {
        return "No Purchase To Cancel.";
    }
    if (this.artworkName == null || !this.artworkName.equals(artworkName)) {
        return "Art Work Name Mismatch.";
    }
    refundableAmount = artworkPrice * 0.10; // 10% refund
    rewardPoints -= finalPrice * 5;
    cancelCount++;
    buyCount--;
    this.cancellationReason = cancellationReason;
    isBought = false;
    this.artworkName = null;
    artworkPrice = 0;
    discountAmount = 0;
    finalPrice = 0;
    return "Your Purchase Is Cancelled. Your Refund Amount = Rs." + refundableAmount;
}
```

Figure 46: Logical Refund Calculation (Standard Cancel)

9. Conclusion

In conclusion, the Art Gallery Management System was successfully designed and implemented to manage visitors, ticketing, artwork purchases, cancellations, billing, discounts, and rewards in a modern and user-friendly way. The project made effective use of Java Swing and OOP principles, ensuring that each component from visitor registration to artwork management was handled with proper encapsulation, validation, and structured logic. Through this work, I was able to strengthen my technical knowledge of GUI development, event handling, and file storage, while also learning the value of defensive programming and error handling for preventing issues such as invalid inputs or Null Pointer Exception. The process was not without challenges, as I initially faced difficulties in saving and loading visitor data from files, supporting multiple artwork purchases per visitor, and implementing precise cancellation features. However, these difficulties provided important learning opportunities, and were addressed through debugging, code restructuring, and adding clear validation checks. By refining the system step by step and focusing on reliability and scalability, the outcome is a robust application that balances both functionality and usability, while reflecting the practical application of core programming concepts.

10. References

- Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.
- Oracle. (n.d.). *The Java™ Tutorials: Creating a GUI With Swing*. Oracle.
<https://docs.oracle.com/javase/tutorial/uiswing/>
- Eckel, B. (2006). *Thinking in Java* (4th ed.). Prentice Hall.
- Deitel, P. J., & Deitel, H. M. (2017). *Java: How to Program* (11th ed.). Pearson.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Oracle. (n.d.). *Class JDialog (Java Platform SE 8)*. Oracle.
<https://docs.oracle.com/javase/8/docs/api/javax/swing/JDialog.html>

- GeeksforGeeks. (2023). *Swing in Java*. GeeksforGeeks. <https://www.geeksforgeeks.org/swing-in-java/>
- Liang, Y. D. (2019). *Introduction to Java Programming and Data Structures: Comprehensive Version* (12th ed.). Pearson.

11. Appendix

```
public abstract class ArtGalleryVisitor{

    // Visitor identity and contact information
    protected int visitorId;
    protected String fullName;
    protected String gender;
    protected String contactNumber;
    protected String registrationDate;

    // Ticket information
    protected double ticketCost;
    protected String ticketType;

    // Visit tracking
    protected int visitCount;
    protected boolean isActive;

    // Purchase tracking
    protected boolean isBought;
```

CS4001NI
Programming

```
protected int buyCount;

protected String artworkName;

protected double artworkPrice;


// Discount and final pricing
protected double finalPrice;
protected double discountAmount;


// Reward points
protected double rewardPoints;


// Cancellation policy
protected final int cancelLimit = 3;
protected int cancelCount;
protected String cancellationReason;
protected double refundableAmount;


// Constructor initializing all visitor attributes and defaults.
public ArtGalleryVisitor(int visitorId, String fullName, String gender, String
contactNumber, String registrationDate, double ticketCost, String ticketType){

    this.visitorId = visitorId;

    this.fullName = fullName;

    this.gender = gender;

    this.contactNumber = contactNumber;

    this.registrationDate = registrationDate;

    this.ticketCost = ticketCost;

    this.ticketType = ticketType;
```

CS4001NI
Programming

```
    this.visitCount = 0;
    this.rewardPoints = 0;
    this.cancelCount = 0;
    this.cancellationReason = "";
    this.refundableAmount = 0;
    this.isActive = false;
    this.isBought = false;
    this.buyCount = 0;
    this.discountAmount = 0;
    this.finalPrice = 0;
    this.artworkName = null;
    this.artworkPrice = 0;
}
```

// Getters

```
public int getVisitorId(){
    return visitorId;
}
```

```
public String getFullName(){
    return fullName;
}
```

```
public String getGender(){
    return gender;
}
```

```
public String getContactNumber(){  
    return contactNumber;  
}
```

```
public String getRegistrationDate(){  
    return registrationDate;  
}
```

```
public double getTicketCost(){  
    return ticketCost;  
}
```

```
public String getTicketType(){  
    return ticketType;  
}
```

```
public int getVisitCount() {  
    return visitCount;  
}
```

```
public double getRewardPoints() {  
    return rewardPoints;  
}
```

```
public int getCancelLimit() {  
    return cancelLimit;  
}
```

```
public int getCancelCount() {  
    return cancelCount;  
}
```

```
public String getCancellationReason() {  
    return cancellationReason;  
}
```

```
public double getRefundableAmount() {  
    return refundableAmount;  
}
```

```
public boolean isActive() {  
    return isActive;  
}
```

```
public boolean isBought() {  
    return isBought;  
}
```

```
public int getBuyCount() {  
    return buyCount;  
}
```

```
public double getFinalPrice() {  
    return finalPrice;  
}
```

```
}
```

```
public double getDiscountAmount() {  
    return discountAmount;
```

```
}
```

```
public String getArtworkName() {  
    return artworkName;  
}
```

```
public double getArtworkPrice() {  
    return artworkPrice;  
}
```

```
public void setFullName(String fullName){  
    this.fullName = fullName;  
}
```

```
public void setGender(String gender){  
    this.gender = gender;  
}
```

```
public void setContactNumber(String contactNumber){  
    this.contactNumber = contactNumber;  
}
```

```
// Log a gallery visit, incrementing the count and activating the account.
```

```
public void logVisit() {  
    visitCount++;  
    isActive = true;  
}
```

```
// Abstract Methods
```

```
public abstract String buyProduct (int visitorID, String artworkName, double  
artworkPrice);
```

```
public abstract double calculateDiscount();
```

```
public abstract double calculateRewardPoint();
```

```
public abstract String cancelProduct (int visitorID, String artworkName, String  
cancellationReason);
```

```
public abstract void generateBill();
```

```
// Display all visitor details, including visits, purchases, and cancellations.
```

```
public void display(){  
    System.out.println("=== Visitor Information ===");  
    System.out.println("ID: " + visitorId);  
    System.out.println("Name: " + fullName);  
    System.out.println("Gender: " + gender);  
    System.out.println("Contact Number: " + contactNumber);  
    System.out.println("Registration Date: " + registrationDate);  
    System.out.println("=== Ticket Information ===");
```

```
        System.out.println("Ticket Type: " + ticketType);
        System.out.println("Ticket Cost: Nrs." + ticketCost);
        System.out.println("Visit Count: " + visitCount);
        System.out.println("=== Purchase Status ===");
        System.out.println("Active Status: " + (isActive ? "Yes" : "No"));
        System.out.println("Purchase Status: " + (isBought ? "Purchased" :
"NotPurchased"));
        System.out.println("Buy Count: " + buyCount);
        System.out.println("Final Price: NRS." + finalPrice);
        System.out.println("Discount Amount: NRS." + discountAmount);
        System.out.println("=== Artwork Details ===");
        System.out.println("Artwork Name: " + (artworkName != null ? artworkName
:"None"));
        System.out.println("Artwork Price: NRS." + artworkPrice);
        System.out.println("=== Rewards ===");
        System.out.println("Reward Points: " + rewardPoints);
        System.out.println("=== Cancellation & Refund ===");
        System.out.println("Cancel Count: " + cancelCount + " (Limit: " + cancelLimit + ")");
        System.out.println("Cancellation Reason: " + (cancellationReason.isEmpty() ?
"None" : cancellationReason));
        System.out.println("Refundable Amount: Nrs." + refundableAmount);
        System.out.println("=====");

    }
}
```

// Concrete implementation of ArtGalleryVisitor for standard-ticket holders.

```
public class StandardVisitor extends ArtGalleryVisitor{

    private boolean isEligibleForDiscountUpgrade;

    private final int visitLimit = 5;

    private float discountPercent;


    // Constructor initializes base visitor info and default discounts.

    public StandardVisitor(int visitorId, String fullName, String gender, String
contactNumber, String registrationDate, double ticketCost, String ticketType){

        super(visitorId, fullName, gender, contactNumber, registrationDate, ticketCost,
ticketType);

        this.discountPercent = 0.10f;

        this.isEligibleForDiscountUpgrade = false;

    }


    // Getters

    public boolean getisEligibleForDiscountUpgrade(){

        return isEligibleForDiscountUpgrade;

    }


    public int getVisitLimit(){

        return visitLimit;

    }


    public float getDiscountPercent(){

        return discountPercent;

    }


    // Check and apply a discount upgrade if visitCount >= visitLimit.
```

```
public boolean checkDiscountUpgrade(){  
    if (visitCount >= visitLimit){  
        isEligibleForDiscountUpgrade = true;  
        discountPercent = 0.15f;  
    }  
    return isEligibleForDiscountUpgrade;  
}
```

// Method to Buy an Art Work.

@Override

```
public String buyProduct(int visitorID, String artworkName, double artworkPrice){  
    if (visitorID != this.visitorId) {  
        return "Visitor ID mismatch. Purchase denied.";  
    }  
    if (!isActive){  
        return "Please log a visit first.";  
    }  
    if (isBought && this.artworkName.equals(artworkName)){  
        return "Art Work Has Already Been Purchased.";  
    }  
    if (artworkPrice <= 0) {  
        return "Invalid artwork price.";  
    }  
    this.artworkName = artworkName;  
    this.artworkPrice = artworkPrice;  
    isBought = true;  
    buyCount++;  
}
```

```
        return "Purchase Successful: " + artworkName;  
    }
```

// Method to Calculate Discount.

@Override

```
public double calculateDiscount(){  
    if (!isBought){  
        return 0;  
    }  
    checkDiscountUpgrade();  
    discountAmount = artworkPrice * discountPercent;  
    finalPrice = artworkPrice - discountAmount;  
    return discountAmount;  
}
```

// Method to Calculate Reward Point.

@Override

```
public double calculateRewardPoint() {  
    if (!isBought) {  
        return 0;  
    }  
    if (finalPrice <= 0) {  
        calculateDiscount();  
    }  
    rewardPoints += finalPrice * 5;  
    return rewardPoints;  
}
```

```
// Method to Cancel a Purchase.

@Override

public String cancelProduct(int visitorID, String artworkName, String
cancellationReason) {
    if (visitorID != this.visitorId) {
        return "Visitor ID mismatch. Cancellation denied.";
    }
    if (cancelCount >= cancelLimit) {
        terminateVisitor();
        return "Cancel Limit Reached. Account has been terminated.";
    }
    if (buyCount == 0 || !isBought) {
        return "No Purchase To Cancel.";
    }
    if (this.artworkName == null || !this.artworkName.equals(artworkName)) {
        return "Art Work Name Mismatch.";
    }
    refundableAmount = artworkPrice * 0.10; // 10% refund
    rewardPoints -= finalPrice * 5;
    cancelCount++;
    buyCount--;
    this.cancellationReason = cancellationReason;
    isBought = false;
    this.artworkName = null;
    artworkPrice = 0;
    discountAmount = 0;
```

```
        finalPrice = 0;

        return "Your Purchase Is Cancelled. Your Refund Amount = Rs." +
refundableAmount;

    }
```

// Method to terminate account and reset counter.

```
private void terminateVisitor() {

    isActive = false;

    isEligibleForDiscountUpgrade = false;

    visitCount = 0;

    cancelCount = 0;

    rewardPoints = 0;

    isBought = false;

    buyCount = 0;

    artworkName = null;

    artworkPrice = 0;

    discountAmount = 0;

    finalPrice = 0;

    refundableAmount = 0;

}
```

@Override

```
public void generateBill(){

    if (buyCount > 0){

        System.out.println("===== Art Gallery Bill =====");

        System.out.println("-----");

        System.out.println("Visitor ID: " + visitorId);

    }

}
```

```
        System.out.println("Visitor Name: " + fullName);
        System.out.println("-----");
        System.out.println("Artwork Details:");
        System.out.println("Name: " + artworkName);
        System.out.println("Original Price: Rs." + artworkPrice);
        System.out.println("Discount Amount: Rs." + discountAmount);
        System.out.println("-----");
        System.out.println("Final Price: Rs." + finalPrice);
        System.out.println("=====");
    }
    else{
        System.out.println("No purchase to bill.");
    }
}

// Display both inherited and StandardVisitor-specific fields.
@Override
public void display(){
    super.display();
    System.out.println("=== Standard Visitor Specific Details ===");
    System.out.println("Eligible for Discount Upgrade: " +
(isEligibleForDiscountUpgrade ? "Yes" : "No"));
    System.out.println("Visit Limit: " + visitLimit);
    System.out.println("Discount Percentage: " + discountPercent + "%");
    System.out.println("=====");
}
}
```

```
// Concrete implementation of ArtGalleryVisitor for elite-ticket holders.
public class EliteVisitor extends ArtGalleryVisitor{
    private boolean assignedPersonalArtAdvisor;
    private boolean exclusiveEventAccess;

    // Constructor initializes base visitor info and elite-specific features.
    public EliteVisitor(int visitorId, String fullName, String gender, String contactNumber,
String registrationDate, double ticketCost, String ticketType){
        super(visitorId, fullName, gender, contactNumber, registrationDate, ticketCost,
ticketType);
        this.assignedPersonalArtAdvisor = false;
        this.exclusiveEventAccess = false;
    }

    // Getters
    public boolean getIsAssignedPersonalArtAdvisor(){
        return assignedPersonalArtAdvisor;
    }

    public boolean getIsExclusiveEventAccess(){
        return exclusiveEventAccess;
    }

    // Assigns a personal art advisor if rewardPoints exceed threshold.
    public boolean assignPersonalArtAdvisor(){
```

```
        if (rewardPoints > 5000){
            assignedPersonalArtAdvisor = true;
        }
        return assignedPersonalArtAdvisor;
    }

    // Grants exclusive access if advisor is assigned.
    public boolean checkExclusiveAccess(){
        if (assignedPersonalArtAdvisor){
            exclusiveEventAccess = true;
        }
        return exclusiveEventAccess;
    }

    // Method to Buy an Art Work.
    @Override
    public String buyProduct(int visitorID, String artworkName, double artworkPrice) {
        if (visitorID != this.visitorId) {
            return "Visitor ID mismatch. Purchase denied.";
        }
        if (!isActive){
            return "Please log a visit first.";
        }
        if (isBought && this.artworkName.equals(artworkName)){
            return "Art Work Has Already Been Purchased.";
        }
        if (artworkPrice <= 0) {
```

```
        return "Invalid artwork price.";
    }
    this.artworkName = artworkName;
    this.artworkPrice = artworkPrice;
    isBought = true;
    buyCount++;
    return "Purchase successful: " + artworkName;
}
```

// Method to Calculate Discount.

```
@Override
public double calculateDiscount() {
    if (!isBought){
        return 0;
    }
    discountAmount = artworkPrice * 0.40;
    finalPrice = artworkPrice - discountAmount;
    return discountAmount;
}
```

// Method to Calculate Reward Point.

```
@Override
public double calculateRewardPoint() {
    if (!isBought) {
        return 0;
    }
    if (finalPrice <= 0) {
```

```
        calculateDiscount(); // ensure discount applied first
    }

    rewardPoints = finalPrice * 10; // reset, not +=
    return rewardPoints;
}
```

// Method to Cancel a Purchase.

@Override

```
public String cancelProduct(int visitorID, String artworkName, String
cancellationReason) {
    if (visitorID != this.visitorId) {
        return "Visitor ID mismatch. Cancellation denied.";
    }

    if (cancelCount >= cancelLimit) {
        terminateVisitor();
        return "Cancel Limit Reached. Account has been terminated.";
    }

    if (buyCount == 0 || !isBought) {
        return "No Purchase To Cancel.";
    }

    if (this.artworkName == null || !this.artworkName.equals(artworkName)) {
        return "Art Work Name Mismatch.";
    }

    refundableAmount = artworkPrice * 0.95; // 95% refund
    rewardPoints = 0; // clear points since purchase cancelled
    cancelCount++;
    buyCount--;
}
```

```
        this.cancellationReason = cancellationReason;
        isBought = false;
        this.artworkName = null;
        artworkPrice = 0;
        discountAmount = 0;
        finalPrice = 0;
        return "Cancelled, refund = Rs." + refundableAmount;
    }
```

// Method to terminate account and reset counter.

```
private void terminateVisitor() {
    assignedPersonalArtAdvisor = false;
    exclusiveEventAccess = false;
    isActive = false;
    visitCount = 0;
    cancelCount = 0;
    rewardPoints = 0;
    isBought = false;
    buyCount = 0;
    artworkName = null;
    artworkPrice = 0;
    discountAmount = 0;
    finalPrice = 0;
    refundableAmount = 0;
}
```

@Override

```
public void generateBill() {  
    if (buyCount > 0) {  
        System.out.println("===== Art Gallery Bill =====");  
        System.out.println("-----");  
        System.out.println("Visitor ID: " + visitorId);  
        System.out.println("Visitor Name: " + fullName);  
        System.out.println("-----");  
        System.out.println("Artwork Details:");  
        System.out.println("Name: " + artworkName);  
        System.out.println("Original Price: Rs." + artworkPrice);  
        System.out.println("Discount Amount: Rs." + discountAmount);  
        System.out.println("-----");  
        System.out.println("Final Price: Rs." + finalPrice);  
        System.out.println("=====");  
    } else {  
        System.out.println("No purchase to bill.");  
    }  
}
```

// Display both inherited and EliteVisitor-specific fields.

@Override

```
public void display(){  
    super.display();  
    System.out.println("=== Elite Visitor Specific Details ===");  
    System.out.println("Personal Art Advisor Assigned: " +  
(assignedPersonalArtAdvisor ? "Yes" : "No"));  
    System.out.println("Exclusive Event Access: " + (exclusiveEventAccess ? "Yes" :  
"No"));
```

```
        System.out.println("=====");  
    }  
}
```

```
import javax.swing.*;  
import java.awt.*;  
import javax.swing.table.DefaultTableModel;  
import javax.swing.table.JTableHeader;  
import java.awt.event.*;  
import java.util.ArrayList;  
import javax.swing.border.TitledBorder;  
import javax.swing.JFileChooser;  
import java.io.File;  
import java.io.PrintWriter;  
import java.io.OutputStreamWriter;  
import java.io.FileOutputStream;  
import java.nio.charset.StandardCharsets;  
import java.io.BufferedReader;  
import java.io.InputStreamReader;  
import java.io.FileInputStream;  
import java.nio.charset.StandardCharsets;
```

```
public class ArtGalleryGUI extends JFrame {  
    private CardLayout cardLayout;  
    private JPanel mainPanel;  
    ArrayList<ArtGalleryVisitor> visitorList = new ArrayList<>();
```

```
private ArrayList<String[]> artworkDataList = new ArrayList<>();  
private String lastVisitorType = "Standard";  
  
public ArtGalleryGUI() {  
    setTitle("AB Art Gallery");  
    setSize(1000, 700);  
    setDefaultCloseOperation(EXIT_ON_CLOSE);  
    setLocationRelativeTo(null);  
  
    cardLayout = new CardLayout();  
    mainPanel = new JPanel(cardLayout);  
  
    JPanel navBar = createNavBar();  
  
    mainPanel.add(createDashboardPanel(), "Dashboard");  
    mainPanel.add(createLoginPanel(), "Login");  
    mainPanel.add(createVisitorPanel(), "Visitors");  
    mainPanel.add(createArtworkPanel(), "Artworks");  
  
    setLayout(new BorderLayout());  
    add(navBar, BorderLayout.NORTH);  
    add(mainPanel, BorderLayout.CENTER);  
  
    cardLayout.show(mainPanel, "Dashboard");  
    setVisible(true);  
}
```

```
private JPanel createNavBar() {  
    JPanel navBar = new JPanel();  
    navBar.setBackground(new Color(1,1,1));  
    navBar.setLayout(new FlowLayout(FlowLayout.CENTER, 20, 10));  
  
    String[] pages = {"Dashboard", "Login", "Visitors", "Artworks"};  
    for (int i = 0; i < pages.length; i++) {  
        final String page = pages[i];  
        JButton btn = new JButton(page);  
        btn.setFocusPainted(false);  
        btn.setFont(new Font("Brush Script MT", Font.PLAIN, 14));  
        btn.setForeground(Color.WHITE);  
        btn.setBackground(new Color(24, 24, 24));  
        btn.addActionListener(new ActionListener() {  
            public void actionPerformed(ActionEvent e) {  
                cardLayout.show(mainPanel, page);  
            }  
        });  
        navBar.add(btn);  
    }  
  
    return navBar;  
}
```

```
private JPanel createDashboardPanel() {  
    JPanel panel = new JPanel() {  
        protected void paintComponent(Graphics g) {
```

```
        super.paintComponent(g);

        ImageIcon bg = new ImageIcon("aj1.jpg");
        g.drawImage(bg.getImage(), 0, 0, getWidth(), getHeight(), this);
    }

};

panel.setLayout(new GridBagLayout());
JLabel label = new JLabel("Welcome to the AB Art Gallery");
label.setFont(new Font("Vivaldi", Font.PLAIN, 40));
label.setForeground(Color.WHITE);
panel.add(label);
return panel;
}
```

```
private JPanel createLoginPanel() {
    JPanel panel = new JPanel(new GridBagLayout());
    panel.setBackground(new Color(1, 1, 1));
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(10, 10, 10, 10);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    Font scriptFont = new Font("Script MT Bold", Font.PLAIN, 14);

    JLabel lblId = new JLabel("Visitor ID:");
    JLabel lblName = new JLabel("Full Name:");
    JLabel lblGender = new JLabel("Gender:");
    JLabel lblContact = new JLabel("Contact:");
    JLabel lblDate = new JLabel("Date (DD/MM/YYYY):");
}
```

```
JLabel lblType = new JLabel("Ticket Type:");
JLabel lblPrice = new JLabel("Ticket Price:");

JLabel[] labels = {lblId, lblName, lblGender, lblContact, lblDate, lblType, lblPrice};
for (JLabel lbl : labels) {
    lbl.setFont(scriptFont);
    lbl.setForeground(Color.WHITE);
}

JTextField txtId = new JTextField();
JTextField txtName = new JTextField();
JTextField txtContact = new JTextField();
JTextField txtPrice = new JTextField("1000");
txtPrice.setEditable(false);
txtPrice.setForeground(Color.WHITE);

JTextField[] fields = {txtId, txtName, txtContact, txtPrice};
for (JTextField field : fields) {
    field.setFont(scriptFont);
    field.setBackground(Color.BLACK);
    field.setForeground(Color.WHITE);
    field.setCaretColor(Color.WHITE);
    field.setBorder(BorderFactory.createLineBorder(Color.GRAY));
}

JRadioButton rbM = new JRadioButton("Male");
JRadioButton rbF = new JRadioButton("Female");
```

```
JRadioButton rbO = new JRadioButton("Other");  
ButtonGroup genderGroup = new ButtonGroup();  
JRadioButton[] radios = {rbM, rbF, rbO};  
for (JRadioButton rb : radios) {  
    rb.setFont(scriptFont);  
    rb.setBackground(Color.BLACK);  
    rb.setForeground(Color.WHITE);  
    genderGroup.add(rb);  
}
```

```
JComboBox<String> cbDay = new JComboBox<>();  
JComboBox<String> cbMonth = new JComboBox<>();  
JComboBox<String> cbYear = new JComboBox<>();  
for (int i = 1; i <= 31; i++) cbDay.addItem(String.valueOf(i));  
String[] months = {"Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep",  
"Oct", "Nov", "Dec"};  
for (String month : months) cbMonth.addItem(month);  
for (int i = 2020; i <= 2035; i++) cbYear.addItem(String.valueOf(i));  
JComboBox[] combos = {cbDay, cbMonth, cbYear};  
for (JComboBox combo : combos) {  
    combo.setFont(scriptFont);  
    combo.setBackground(Color.BLACK);  
    combo.setForeground(Color.WHITE);  
}
```

```
JPanel genderPanel = new JPanel();  
genderPanel.setBackground(panel.getBackground());
```

```
genderPanel.add(rbM);
genderPanel.add(rbF);
genderPanel.add(rbO);

JPanel datePanel = new JPanel();
datePanel.setBackground(panel.getBackground());
datePanel.add(cbDay);
datePanel.add(cbMonth);
datePanel.add(cbYear);

JComboBox<String> cbType = new JComboBox<>(new String[]{"Standard",
"Elite"});
cbType.setFont(scriptFont);
cbType.setBackground(Color.BLACK);
cbType.setForeground(Color.WHITE);
cbType.addItemListener(new ItemListener() {
    public void itemStateChanged(ItemEvent e) {
        if (e.getStateChange() == ItemEvent.SELECTED) {
            txtPrice.setText(cbType.getSelectedItem().equals("Elite") ? "2000" :
"1000");
            txtPrice.setForeground(Color.WHITE);
        }
    }
});

JButton btnAdd = createNavButton("Add Visitor");
JButton btnClear = createNavButton("Clear");
JButton btnLogVisit = createNavButton("Log Visit");
```

```

JButton[] buttons = {btnAdd, btnClear, btnLogVisit};
for (JButton btn : buttons) {
    btn.setFont(scriptFont);
    btn.setBackground(new Color(24, 24, 24));
    btn.setForeground(Color.WHITE);
    btn.setFocusPainted(false);
    btn.setPreferredSize(new Dimension(120, 30));
}

JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 20, 10));
buttonPanel.setBackground(panel.getBackground());
buttonPanel.add(btnAdd);
buttonPanel.add(btnClear);
buttonPanel.add(btnLogVisit);

btnAdd.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            String idStr = txtId.getText().trim();
            String name = txtName.getText().trim();
            String contact = txtContact.getText().trim();

            // === Required Field Check ===

            if (idStr.isEmpty() || name.isEmpty() || contact.isEmpty() ||
genderGroup.getSelection() == null) {
                JOptionPane.showMessageDialog(panel, "Please fill in all required
fields.", "Validation Error", JOptionPane.WARNING_MESSAGE);

```

```
        return;
    }

    // === Validate ID ===
    for (char c : idStr.toCharArray()) {
        if (!Character.isDigit(c)) {
            throw new IllegalArgumentException("Visitor ID must contain only
numbers.");
        }
    }
    int id = Integer.parseInt(idStr);

    // === Validate Name ===
    for (char c : name.toCharArray()) {
        if (Character.isDigit(c)) {
            throw new IllegalArgumentException("Full Name must not contain
numbers.");
        }
    }

    // === Validate Contact ===
    for (char c : contact.toCharArray()) {
        if (!Character.isDigit(c)) {
            throw new IllegalArgumentException("Contact number must contain
only numbers.");
        }
    }
    if (contact.length() != 10) {
```

```
        throw new IllegalArgumentException("Contact number must be exactly  
10 digits.");  
    }  
  
    // === Check for Duplicate ID ===  
    for (ArtGalleryVisitor v : visitorList) {  
        if (v.getVisitorId() == id) {  
            JOptionPane.showMessageDialog(panel, "Visitor ID already exists.  
Duplicate not allowed.", "Duplicate Visitor", JOptionPane.WARNING_MESSAGE);  
            return;  
        }  
    }  
  
    // === Prepare Remaining Fields ===  
    String gender = rbM.isSelected() ? "Male" : rbF.isSelected() ? "Female" :  
"Other";  
  
    String date = cbDay.getSelectedItem() + "/" + cbMonth.getSelectedItem()  
+ "/" + cbYear.getSelectedItem();  
  
    String type = (String) cbType.getSelectedItem();  
    double ticketCost = type.equals("Elite") ? 2000 : 1000;  
  
    // === Create Visitor Object ===  
    ArtGalleryVisitor visitor = type.equals("Elite")  
        ? new EliteVisitor(id, name, gender, contact, date, ticketCost, type)  
        : new StandardVisitor(id, name, gender, contact, date, ticketCost,  
type);  
  
    visitorList.add(visitor);  
    lastVisitorType = type;
```

```
        JOptionPane.showMessageDialog(panel, "Visitor added successfully!",  
"Success", JOptionPane.INFORMATION_MESSAGE);  
        saveVisitorsToFileTables();  
  
    } catch (IllegalArgumentException ex) {  
        JOptionPane.showMessageDialog(panel, ex.getMessage(), "Input  
Error", JOptionPane.ERROR_MESSAGE);  
    } catch (Exception ex) {  
        JOptionPane.showMessageDialog(panel, "Error: " + ex.getMessage(),  
"Error", JOptionPane.ERROR_MESSAGE);  
    }  
}  
});  
  
btnClear.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        txtId.setText("");  
        txtName.setText("");  
        txtContact.setText("");  
        genderGroup.clearSelection();  
        cbDay.setSelectedIndex(0);  
        cbMonth.setSelectedIndex(0);  
        cbYear.setSelectedIndex(0);  
        cbType.setSelectedIndex(0);  
        txtPrice.setText("1000");  
    }  
});
```

```
btnLogVisit.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        String idStr = txtId.getText().trim();  
        String name = txtName.getText().trim();  
        String contact = txtContact.getText().trim();  
  
        if (idStr.isEmpty() || name.isEmpty() || contact.isEmpty() ||  
genderGroup.getSelection() == null) {  
            JOptionPane.showMessageDialog(panel, "Please fill in all required  
fields.", "Validation Error", JOptionPane.WARNING_MESSAGE);  
            return;  
        }  
  
        try {  
            for (char c : idStr.toCharArray()) {  
                if (!Character.isDigit(c)) {  
                    throw new IllegalArgumentException("Visitor ID must contain only  
numbers.");  
                }  
            }  
        }  
  
        for (char c : name.toCharArray()) {  
            if (Character.isDigit(c)) {  
                throw new IllegalArgumentException("Full Name must not contain  
numbers.");  
            }  
        }  
    }  
});
```

```
// === Validate Contact ===
for (char c : contact.toCharArray()) {
    if (!Character.isDigit(c)) {
        throw new IllegalArgumentException("Contact number must contain
only numbers.");
    }
}
if (contact.length() != 10) {
    throw new IllegalArgumentException("Contact number must be exactly
10 digits.");
}

int id = Integer.parseInt(idStr);
boolean found = false;
for (ArtGalleryVisitor visitor : visitorList) {
    if (visitor.getVisitorId() == id) {
        visitor.logVisit();
        JOptionPane.showMessageDialog(panel, "Visit logged for visitor ID
" + id +
        "\nTotal visits: " + visitor.getVisitCount(), "Visit Logged",
JOptionPane.INFORMATION_MESSAGE);
        saveVisitorsToFileTables();
        found = true;
        break;
    }
}
if (!found) {
```

```
        JOptionPane.showMessageDialog(panel, "Visitor ID not found.",
"Error", JOptionPane.ERROR_MESSAGE);
    }
    } catch (IllegalArgumentException ex) {
        JOptionPane.showMessageDialog(panel, ex.getMessage(), "Input
Error", JOptionPane.ERROR_MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(panel, "Error: " + ex.getMessage(),
"Error", JOptionPane.ERROR_MESSAGE);
    }
}
});

int y = 0;

gbc.gridx = 0; gbc.gridy = y; panel.add(lblId, gbc); gbc.gridx = 1; panel.add(txtId,
gbc);

gbc.gridx = 0; gbc.gridy = ++y; panel.add(lblName, gbc); gbc.gridx = 1;
panel.add(txtName, gbc);

gbc.gridx = 0; gbc.gridy = ++y; panel.add(lblGender, gbc); gbc.gridx = 1;
panel.add(genderPanel, gbc);

gbc.gridx = 0; gbc.gridy = ++y; panel.add(lblContact, gbc); gbc.gridx = 1;
panel.add(txtContact, gbc);

gbc.gridx = 0; gbc.gridy = ++y; panel.add(lblDate, gbc); gbc.gridx = 1;
panel.add(datePanel, gbc);

gbc.gridx = 0; gbc.gridy = ++y; panel.add(lblType, gbc); gbc.gridx = 1;
panel.add(cbType, gbc);

gbc.gridx = 0; gbc.gridy = ++y; panel.add(lblPrice, gbc); gbc.gridx = 1;
panel.add(txtPrice, gbc);

gbc.gridx = 0; gbc.gridy = ++y; gbc.gridwidth = 2; panel.add(buttonPanel, gbc);
```

```
    return panel;  
}
```

```
private JPanel createVisitorPanel() {
```

```
    JPanel root = new JPanel(new BorderLayout(10, 10));
```

```
    root.setBackground(Color.BLACK);
```

```
    root.setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));
```

```
    JPanel tablePanel = new JPanel(new GridLayout(1, 2, 8, 0));
```

```
    tablePanel.setBackground(Color.BLACK);
```

```
    // --- columns with status ---
```

```
    String[] stdCols =
```

```
{"ID","Name","Gender","Contact","Date","Type","Price","Visits","Upgrade?","Discount  
%"};
```

```
    String[] eliteCols =
```

```
{"ID","Name","Gender","Contact","Date","Type","Price","Visits","Advisor","Event  
Access"};
```

```
    // --- Standard table ---
```

```
    final DefaultTableModel stdModel = new DefaultTableModel(stdCols, 0);
```

```
    JTable stdTable = new JTable(stdModel);
```

```
    stdTable.setFillViewportHeight(true);
```

```
    stdTable.setBackground(Color.BLACK);
```

```
    stdTable.setForeground(Color.WHITE);
```

```
    stdTable.setFont(new Font("Script MT Bold", Font.PLAIN, 14));
```

```
    stdTable.setRowHeight(24);
```

```
    JTableHeader sh = stdTable.getTableHeader();
```

```
sh.setFont(new Font("Script MT Bold", Font.PLAIN, 14));
sh.setBackground(Color.BLACK); sh.setForeground(Color.WHITE);
sh.setReorderingAllowed(false);
JScrollPane stdScroll = new JScrollPane(stdTable);
stdScroll.getViewport().setBackground(Color.BLACK);

JPanel stdColPanel = new JPanel(new BorderLayout(0, 6));
stdColPanel.setBackground(Color.BLACK);
JLabel stdTitle = new JLabel("Standard Visitors");
stdTitle.setFont(new Font("Script MT Bold", Font.PLAIN, 16));
stdTitle.setForeground(Color.WHITE);
stdColPanel.add(stdTitle, BorderLayout.NORTH);
stdColPanel.add(stdScroll, BorderLayout.CENTER);

// --- Elite table ---
final DefaultTableModel eliteModel = new DefaultTableModel(eliteCols, 0);
JTable eliteTable = new JTable(eliteModel);
eliteTable.setFillViewportHeight(true);
eliteTable.setBackground(Color.BLACK);
eliteTable.setForeground(Color.WHITE);
eliteTable.setFont(new Font("Script MT Bold", Font.PLAIN, 14));
eliteTable.setRowHeight(24);
JTableHeader eh = eliteTable.getTableHeader();
eh.setFont(new Font("Script MT Bold", Font.PLAIN, 14));
eh.setBackground(Color.BLACK); eh.setForeground(Color.WHITE);
eh.setReorderingAllowed(false);
JScrollPane eliteScroll = new JScrollPane(eliteTable);
```

```
eliteScroll.getViewport().setBackground(Color.BLACK);

JPanel eliteColPanel = new JPanel(new BorderLayout(0, 6));
eliteColPanel.setBackground(Color.BLACK);
JLabel eliteTitle = new JLabel("Elite Visitors");
eliteTitle.setFont(new Font("Script MT Bold", Font.PLAIN, 16));
eliteTitle.setForeground(Color.WHITE);
eliteColPanel.add(eliteTitle, BorderLayout.NORTH);
eliteColPanel.add(eliteScroll, BorderLayout.CENTER);

tablePanel.add(stdColPanel);
tablePanel.add(eliteColPanel);

// --- Controls ---
JPanel buttonPanel = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 8));
buttonPanel.setBackground(Color.BLACK);

JLabel idLbl = new JLabel("Visitor ID:");
idLbl.setForeground(Color.WHITE);
idLbl.setFont(new Font("Script MT Bold", Font.PLAIN, 14));

Font scriptFont = new Font("Brush Script MT", Font.PLAIN, 16);
final JTextField idField = new JTextField(10);
idField.setFont(scriptFont);
idField.setBackground(Color.BLACK);
idField.setForeground(Color.WHITE);
idField.setCaretColor(Color.WHITE);
```

```
idField.setBorder(BorderFactory.createLineBorder(Color.GRAY));
```

```
JButton assignAdvisorBtn = new JButton("Assign Personal Art Advisor");  
assignAdvisorBtn.setFont(new Font("Brush Script MT", Font.PLAIN, 18));  
assignAdvisorBtn.setBackground(new Color(24, 24, 24));  
assignAdvisorBtn.setForeground(Color.WHITE);  
assignAdvisorBtn.setFocusPainted(false);  
assignAdvisorBtn.setPreferredSize(new Dimension(120, 30));
```

```
JButton checkUpgradeBtn = new JButton("Check Upgrade");  
checkUpgradeBtn.setFont(new Font("Brush Script MT", Font.PLAIN, 18));  
checkUpgradeBtn.setBackground(new Color(24, 24, 24));  
checkUpgradeBtn.setForeground(Color.WHITE);  
checkUpgradeBtn.setFocusPainted(false);  
checkUpgradeBtn.setPreferredSize(new Dimension(120, 30));
```

```
JButton clearBtn = new JButton("Clear");  
clearBtn.setFont(new Font("Brush Script MT", Font.PLAIN, 18));  
clearBtn.setBackground(new Color(24, 24, 24));  
clearBtn.setForeground(Color.WHITE);  
clearBtn.setFocusPainted(false);  
clearBtn.setPreferredSize(new Dimension(120, 30));
```

```
clearBtn.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        idField.setText("");  
    }  
})
```

```
});
```

```
 JButton readFileBtn = new JButton("Read File");  
 readFileBtn.setFont(new Font("Brush Script MT", Font.PLAIN, 18));  
 readFileBtn.setBackground(new Color(24,24,24));  
 readFileBtn.setForeground(Color.WHITE);  
 readFileBtn.setFocusPainted(false);  
 readFileBtn.setPreferredSize(new Dimension(140, 30));  
 buttonPanel.add(readFileBtn);
```

```
 buttonPanel.add(idLbl);  
 buttonPanel.add(idField);  
 buttonPanel.add(assignAdvisorBtn);  
 buttonPanel.add(checkUpgradeBtn);  
 buttonPanel.add(clearBtn);  
 buttonPanel.add(readFileBtn);
```

```
 root.add(tablePanel, BorderLayout.CENTER);  
 root.add(buttonPanel, BorderLayout.SOUTH);
```

```
// --- Inline REFRESH logic (reusable, no lambdas) ---
```

```
final Runnable refresh = new Runnable() {  
    public void run() {  
        stdModel.setRowCount(0);  
        eliteModel.setRowCount(0);  
        for (ArtGalleryVisitor v : visitorList) {  
            if (v instanceof StandardVisitor) {
```

```
StandardVisitor sv = (StandardVisitor) v;

String upgraded = sv.checkDiscountUpgrade() ? "Yes" : "No";
String discPct = String.format("%.0f%%", sv.getDiscountPercent() *
100.0f);

stdModel.addRow(new Object[]{
    v.getVisitorId(), v.getFullName(), v.getGender(),
    v.getContactNumber(), v.getRegistrationDate(),
    v.getTicketType(), v.getTicketCost(), v.getVisitCount(),
    upgraded, discPct
});
} else if (v instanceof EliteVisitor) {
    EliteVisitor ev = (EliteVisitor) v;
    String advisor = ev.assignPersonalArtAdvisor() ? "Yes" : "No";
    String access = ev.checkExclusiveAccess() ? "Yes" : "No";
    eliteModel.addRow(new Object[]{
        v.getVisitorId(), v.getFullName(), v.getGender(),
        v.getContactNumber(), v.getRegistrationDate(),
        v.getTicketType(), v.getTicketCost(), v.getVisitCount(),
        advisor, access
    });
}
}
}
};

// Populate on show
root.addComponentListener(new ComponentAdapter() {
```

```
        public void componentShown(ComponentEvent e) { refresh.run(); }

    });

    // --- Button actions with instanceof + casting + refresh ---
    assignAdvisorBtn.addActionListener(new ActionListener() {

        public void actionPerformed(ActionEvent e) {

            String s = idField.getText().trim();

            if (s.isEmpty()) {

                JOptionPane.showMessageDialog(root, "Enter Visitor ID.", "Error",
JOptionPane.ERROR_MESSAGE);

                return;
            }

            try {

                int id = Integer.parseInt(s);

                boolean found = false;

                for (ArtGalleryVisitor v : visitorList) {

                    if (v.getVisitorId() == id) {

                        found = true;

                        if (v instanceof EliteVisitor) {

                            EliteVisitor ev = (EliteVisitor) v; // cast

                            boolean assigned = ev.assignPersonalArtAdvisor();

                            ev.checkExclusiveAccess();

                            JOptionPane.showMessageDialog(root,

                                assigned ? "Advisor assigned (or already eligible)." : "Not
enough reward points.",

                                "Advisor Status", JOptionPane.INFORMATION_MESSAGE);

                            saveVisitorsToFileTables();

                            refresh.run();
                        }
                    }
                }
            }
        }
    });
}
```

```
        } else {  
            JOptionPane.showMessageDialog(root, "This visitor is not an  
Elite Visitor.",  
                "Type Mismatch", JOptionPane.ERROR_MESSAGE);  
        }  
        break;  
    }  
}  
if (!found) {  
    JOptionPane.showMessageDialog(root, "Visitor ID not found.", "Error",  
JOptionPane.ERROR_MESSAGE);  
}  
} catch (NumberFormatException ex) {  
    JOptionPane.showMessageDialog(root, "Visitor ID must be a number.",  
"Input Error", JOptionPane.ERROR_MESSAGE);  
} catch (Exception ex) {  
    JOptionPane.showMessageDialog(root, "Unexpected error: " +  
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);  
}  
}  
});
```

```
checkUpgradeBtn.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        String s = idField.getText().trim();  
        if (s.isEmpty()) {  
            JOptionPane.showMessageDialog(root, "Enter Visitor ID.", "Error",  
JOptionPane.ERROR_MESSAGE);  
            return;  
        }  
    }  
});
```

```
    }  
    try {  
        int id = Integer.parseInt(s);  
        boolean found = false;  
        for (ArtGalleryVisitor v : visitorList) {  
            if (v.getVisitorId() == id) {  
                found = true;  
                if (v instanceof StandardVisitor) {  
                    StandardVisitor sv = (StandardVisitor) v; // cast  
                    boolean up = sv.checkDiscountUpgrade();  
                    JOptionPane.showMessageDialog(root,  
                        up ? "Eligible for 15% discount." : "Visit limit not reached  
(requires 5).",  
                        "Upgrade Status", JOptionPane.INFORMATION_MESSAGE);  
                    saveVisitorsToFileTables();  
                    refresh.run();  
                } else {  
                    JOptionPane.showMessageDialog(root, "This visitor is not a  
Standard Visitor.",  
                        "Type Mismatch", JOptionPane.ERROR_MESSAGE);  
                }  
                break;  
            }  
        }  
        if (!found) {  
            JOptionPane.showMessageDialog(root, "Visitor ID not found.", "Error",  
JOptionPane.ERROR_MESSAGE);  
        }  
    }  
}
```

```
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(root, "Visitor ID must be a number.",
"Input Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(root, "Unexpected error: " +
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
});

readFileBtn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        File f = new File("visitors.txt");
        if (!f.exists()) {
            JOptionPane.showMessageDialog(
                root, // safe parent (the Visitors panel)
                "visitors.txt not found.",
                "Read File",
                JOptionPane.WARNING_MESSAGE
            );
            return;
        }

        // Build models for the file view

        String[] stdCols =
{"ID", "Name", "Gender", "Contact", "RegDate", "Ticket", "Cost", "Visits", "Discount", "Upgrade
Eligible"};
```

```
String[] eliteCols =  
{"ID","Name","Gender","Contact","RegDate","Ticket","Cost","Visits","ArtAdvisor","EventA  
ccess"};
```

```
DefaultTableModel stdFileModel = new DefaultTableModel(stdCols, 0);  
DefaultTableModel eliteFileModel = new DefaultTableModel(eliteCols, 0);
```

```
boolean inStd = false, inElite = false;  
int skip = 0;
```

```
try (BufferedReader br = new BufferedReader(  
    new InputStreamReader(new FileInputStream(f)))) {
```

```
    String line;
```

```
    while ((line = br.readLine()) != null) {
```

```
        // detect sections
```

```
        if (line.startsWith("==== Standard Visitors")) { inStd = true; inElite =  
false; skip = 2; continue; }
```

```
        if (line.startsWith("==== Elite Visitors")) { inElite = true; inStd = false;  
skip = 2; continue; }
```

```
        // skip header and dashed line
```

```
        if (skip > 0) { skip--; continue; }
```

```
        // ignore blanks and separators
```

```
        if (line.trim().isEmpty() || line.startsWith("-") || line.startsWith("(none)")  
continue;
```

```
        // split on 2+ spaces (your fixed-width table)
```

```
String[] parts = line.trim().split("\\s{2,}");
if (inStd && parts.length >= 10) {
    stdFileModel.addRow(new Object[]{
        parts[0],parts[1],parts[2],parts[3],parts[4],
        parts[5],parts[6],parts[7],parts[8],parts[9]
    });
} else if (inElite && parts.length >= 10) {
    eliteFileModel.addRow(new Object[]{
        parts[0],parts[1],parts[2],parts[3],parts[4],
        parts[5],parts[6],parts[7],parts[8],parts[9]
    });
}
}
}
} catch (Exception ex) {
    JOptionPane.showMessageDialog(
        root,
        "Error reading visitors.txt:\n" + ex.getMessage(),
        "Read File",
        JOptionPane.ERROR_MESSAGE
    );
    return;
}

// ===== Build and show the modal dialog with the two tables =====

Window owner = SwingUtilities.getWindowAncestor(root); // SAFE: 'root' is
a real component
```

```
        JDialog dlg = (owner != null)
            ? new JDialog(owner, "Visitors (from file)",
Dialog.ModalityType.APPLICATION_MODAL)
            : new JDialog((Frame) null, "Visitors (from file)", true); // fallback if owner
is null
```

```
dlg.setDefaultCloseOperation(JDialog.DISPOSE_ON_CLOSE);
```

```
JPanel container = new JPanel(new GridLayout(1, 2, 8, 0));
container.setBackground(Color.BLACK);
container.setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));
```

```
// ----- Standard table UI -----
```

```
JTable stdTable = new JTable(stdFileModel);
stdTable.setFillViewportHeight(true);
stdTable.setBackground(Color.BLACK);
stdTable.setForeground(Color.WHITE);
stdTable.setFont(new Font("Script MT Bold", Font.PLAIN, 14));
stdTable.setRowHeight(24);
JTableHeader stdHeader = stdTable.getTableHeader();
stdHeader.setFont(new Font("Script MT Bold", Font.PLAIN, 14));
stdHeader.setBackground(Color.BLACK);
stdHeader.setForeground(Color.WHITE);
stdHeader.setReorderingAllowed(false);
```

```
JScrollPane stdScroll = new JScrollPane(stdTable);
stdScroll.getViewport().setBackground(Color.BLACK);
```

```
JPanel stdCol = new JPanel(new BorderLayout(0, 6));
stdCol.setBackground(Color.BLACK);
JLabel stdTitle = new JLabel("Standard Visitors (from file)");
stdTitle.setFont(new Font("Script MT Bold", Font.PLAIN, 16));
stdTitle.setForeground(Color.WHITE);
stdCol.add(stdTitle, BorderLayout.NORTH);
stdCol.add(stdScroll, BorderLayout.CENTER);

// ----- Elite table UI -----
JTable eliteTable = new JTable(eliteFileModel);
eliteTable.setFillViewportHeight(true);
eliteTable.setBackground(Color.BLACK);
eliteTable.setForeground(Color.WHITE);
eliteTable.setFont(new Font("Script MT Bold", Font.PLAIN, 14));
eliteTable.setRowHeight(24);
JTableHeader eliteHeader = eliteTable.getTableHeader();
eliteHeader.setFont(new Font("Script MT Bold", Font.PLAIN, 14));
eliteHeader.setBackground(Color.BLACK);
eliteHeader.setForeground(Color.WHITE);
eliteHeader.setReorderingAllowed(false);

JScrollPane eliteScroll = new JScrollPane(eliteTable);
eliteScroll.getViewport().setBackground(Color.BLACK);

JPanel eliteCol = new JPanel(new BorderLayout(0, 6));
eliteCol.setBackground(Color.BLACK);
JLabel eliteTitle = new JLabel("Elite Visitors (from file)");
```

```
        eliteTitle.setFont(new Font("Script MT Bold", Font.PLAIN, 16));
        eliteTitle.setForeground(Color.WHITE);
        eliteCol.add(eliteTitle, BorderLayout.NORTH);
        eliteCol.add(eliteScroll, BorderLayout.CENTER);

        container.add(stdCol);
        container.add(eliteCol);

        dlg.setContentPane(container);
        dlg.setSize(900, 500);
        dlg.setLocationRelativeTo(owner); // centers on main window
        dlg.setVisible(true);
    }
});

return root;
}

private void saveVisitorsToFileTables() {
    try (java.io.PrintWriter out = new java.io.PrintWriter(
        new java.io.FileOutputStream("visitors.txt"))) {

        // ===== Standard Visitors =====
        out.println("===== Standard Visitors =====");
        out.printf("%-5s %-18s %-8s %-12s %-12s %-10s %-8s %-6s %-9s %-16s%n",
            "ID", "Name", "Gender", "Contact", "RegDate", "Ticket", "Cost", "Visits", "Discount", "Upgrade
            Eligible");
```

```
out.println("-----");
```

```
    boolean anyStd = false;
    for (ArtGalleryVisitor v : visitorList) {
        if (v instanceof StandardVisitor) {
            anyStd = true;
            StandardVisitor sv = (StandardVisitor) v;
            String discPct = String.format("%.0f%%", sv.getDiscountPercent() * 100f);
            String eligible = sv.checkDiscountUpgrade() ? "Yes" : "No";

            // inline clipping with ternary
            String name = (v.getFullName() != null && v.getFullName().length() > 18)
? v.getFullName().substring(0,17)+"..." : v.getFullName();

            String contact = (v.getContactNumber() != null &&
v.getContactNumber().length() > 12) ? v.getContactNumber().substring(0,11)+"..." :
v.getContactNumber();

            String regDate = (v.getRegistrationDate() != null &&
v.getRegistrationDate().length() > 12) ? v.getRegistrationDate().substring(0,11)+"..." :
v.getRegistrationDate();

            out.printf("%-5d %-18s %-8s %-12s %-12s %-10s %-8.2f %-6d %-9s
%-16s%n",
                v.getVisitorId(),
                name,
                v.getGender(),
                contact,
                regDate,
                v.getTicketType(),
```

```
        v.getTicketCost(),
        v.getVisitCount(),
        discPct,
        eligible);
    }
}
if (!anyStd) out.println("(none)");
out.println();

// ===== Elite Visitors =====
out.println("===== Elite Visitors =====");
out.printf("%-5s %-18s %-8s %-12s %-12s %-10s %-8s %-6s %-11s %-11s%n",

"ID","Name","Gender","Contact","RegDate","Ticket","Cost","Visits","ArtAdvisor","EventA
ccess");

out.println("-----");

boolean anyElite = false;
for (ArtGalleryVisitor v : visitorList) {
    if (v instanceof EliteVisitor) {
        anyElite = true;
        EliteVisitor ev = (EliteVisitor) v;
        String advisor = ev.assignPersonalArtAdvisor() ? "Yes" : "No";
        String access = ev.checkExclusiveAccess() ? "Yes" : "No";

        String name = (v.getFullName() != null && v.getFullName().length() > 18)
? v.getFullName().substring(0,17)+"..." : v.getFullName();
```

```
        String contact = (v.getContactNumber() != null &&  
v.getContactNumber().length() > 12) ? v.getContactNumber().substring(0,11)+"..." :  
v.getContactNumber();  
  
        String regDate = (v.getRegistrationDate() != null &&  
v.getRegistrationDate().length() > 12) ? v.getRegistrationDate().substring(0,11)+"..." :  
v.getRegistrationDate();  
  
        out.printf("%-5d %-18s %-8s %-12s %-12s %-10s %-8.2f %-6d %-11s  
%-11s%n",  
  
            v.getVisitorId(),  
            name,  
            v.getGender(),  
            contact,  
            regDate,  
            v.getTicketType(),  
            v.getTicketCost(),  
            v.getVisitCount(),  
            advisor,  
            access);  
    }  
}  
  
if (!anyElite) out.println("(none)");  
  
} catch (Exception ex) {  
    javax.swing.JOptionPane.showMessageDialog(  
        null,  
        "Error saving visitors.txt: " + ex.getMessage(),  
        "File Error",  
        javax.swing.JOptionPane.ERROR_MESSAGE
```

```
    );  
    }  
}
```

```
private JPanel createArtworkPanel() {  
    JPanel panel = new JPanel(new BorderLayout());  
    panel.setBackground(new Color(1, 1, 1));  
  
    Font scriptFont = new Font("Script MT Bold", Font.PLAIN, 14);  
  
    JPanel formPanel = new JPanel(new GridBagLayout());  
    formPanel.setBackground(panel.getBackground());  
    GridBagConstraints gbc = new GridBagConstraints();  
    gbc.insets = new Insets(8, 8, 8, 8);  
    gbc.fill = GridBagConstraints.HORIZONTAL;  
  
    JLabel lblVisitorId = new JLabel("Visitor ID:");  
    JLabel lblName = new JLabel("Artwork Name:");  
    JLabel lblPrice = new JLabel("Price:");  
    JLabel lblCancelReason = new JLabel("Cancel Reason:");  
  
    JLabel[] labels = {lblVisitorId, lblName, lblPrice, lblCancelReason};  
    for (JLabel lbl : labels) {  
        lbl.setFont(scriptFont);  
        lbl.setForeground(Color.WHITE);  
    }  
}
```

```
    JTextField txtVisitorId = new JTextField();
    JTextField txtName = new JTextField();
    JTextField txtPrice = new JTextField();
    JTextField txtCancelReason = new JTextField();

    JTextField[] fields = {txtVisitorId, txtName, txtPrice, txtCancelReason};
    for (JTextField txt : fields) {
        txt.setFont(scriptFont);
        txt.setBackground(Color.BLACK);
        txt.setForeground(Color.WHITE);
        txt.setCaretColor(Color.WHITE);
        txt.setBorder(BorderFactory.createLineBorder(Color.GRAY));
    }

    Dimension btnSize = new Dimension(200, 30);

    JButton btnBuy = createNavButton("Buy Artwork");
    JButton btnCancel = createNavButton("Cancel Artwork");
    JButton btnBill = createNavButton("Generate Bill");
    JButton btnClear = createNavButton("Clear");
    JButton btnReward = createNavButton("Calculate Reward Points");
    JButton btnDiscount = createNavButton("Calculate Discount");

    JButton[] buttons = {btnBuy, btnCancel, btnBill, btnClear, btnReward, btnDiscount};
    for (JButton btn : buttons) {
        btn.setFont(scriptFont);
        btn.setPreferredSize(btnSize);
    }
```

```
        btn.setMinimumSize(btnSize);
        btn.setMaximumSize(btnSize);
        btn.setBackground(new Color(24, 24, 24));
        btn.setForeground(Color.WHITE);
        btn.setFocusPainted(false);
    }

    String[] columns = {"ID", "Artwork Name", "Price"};
    DefaultTableModel model = new DefaultTableModel(columns, 0);
    JTable table = new JTable(model);
    table.setBackground(Color.BLACK);
    table.setForeground(Color.WHITE);
    table.setFont(scriptFont);
    table.setRowHeight(24);
    JTableHeader header = table.getTableHeader();
    header.setFont(new Font("Script MT Bold", Font.PLAIN, 12));
    header.setBackground(Color.BLACK);
    header.setForeground(Color.WHITE);

    JScrollPane tableScrollPane = new JScrollPane(table);
    tableScrollPane.getViewport().setBackground(Color.BLACK);

    btnBuy.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            String visitorIdText = txtVisitorId.getText().trim();
            String nameText      = txtName.getText().trim();
            String priceText     = txtPrice.getText().trim();
        }
    });
```

```
        if (visitorIdText.isEmpty() || nameText.isEmpty() || priceText.isEmpty()) {
            JOptionPane.showMessageDialog(panel, "Enter Visitor ID, Artwork
name, and price.",
                "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        try {
            // ---- validate inputs ----
            for (char c : visitorIdText.toCharArray()) if (!Character.isDigit(c))
                throw new IllegalArgumentException("Visitor ID must contain only
digits.");

            int visitorId = Integer.parseInt(visitorIdText);

            for (char c : nameText.toCharArray())
                if (!Character.isLetter(c) && c != ' ')
                    throw new IllegalArgumentException("Artwork name must contain
only letters and spaces.");

            double price = Double.parseDouble(priceText);

            if (price <= 0) throw new IllegalArgumentException("Price must be
greater than 0.");

            // ---- find visitor ----
            ArtGalleryVisitor target = null;
            for (ArtGalleryVisitor v : visitorList) {
                if (v.getVisitorId() == visitorId) { target = v; break; }
            }
        }
    }
}
```

```
    }  
    if (target == null)  
        throw new IllegalArgumentException("Visitor ID not found.");  
  
    // ---- perform purchase ----  
    String msg = target.buyProduct(visitorId, nameText, price);  
    if (!msg.toLowerCase().contains("successful")) {  
        JOptionPane.showMessageDialog(panel, msg, "Buy Artwork",  
JOptionPane.WARNING_MESSAGE);  
        return;  
    }  
  
    // prepare numbers for later screens/bill  
    target.calculateDiscount();    // sets discountAmount & finalPrice  
    target.calculateRewardPoint(); // sets rewardPoints (per purchase  
policy)  
  
    // ---- update table (THIS is what was missing) ----  
    model.addRow(new Object[] { visitorId, nameText, price });  
  
    JOptionPane.showMessageDialog(panel, "Purchase recorded: " +  
nameText,  
        "Buy Artwork", JOptionPane.INFORMATION_MESSAGE);  
  
} catch (NumberFormatException ex) {  
    JOptionPane.showMessageDialog(panel, "Visitor ID and Price must be  
numeric.",  
        "Input Error", JOptionPane.ERROR_MESSAGE);  
}
```

```
        } catch (IllegalArgumentException ex) {
            JOptionPane.showMessageDialog(panel, ex.getMessage(),
                "Input Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(panel, "Unexpected error: " +
ex.getMessage(),
                "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
});

btnCancel.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String visitorIdText = txtVisitorId.getText().trim();
        String reason      = txtCancelReason.getText().trim();
        String nameText    = txtName.getText().trim();
        String priceText   = txtPrice.getText().trim();

        if (visitorIdText.isEmpty() || nameText.isEmpty() || priceText.isEmpty() ||
reason.isEmpty()) {
            JOptionPane.showMessageDialog(panel,
                "Please fill all fields (Visitor ID, Artwork Name, Price, Reason).",
                "Input Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        try {
            // --- validate ID ---
```

```
        for (char c : visitorIdText.toCharArray())
            if (!Character.isDigit(c))
                throw new IllegalArgumentException("Visitor ID must contain only
digits.");

        int visitorId = Integer.parseInt(visitorIdText);

        // --- validate name ---
        for (char c : nameText.toCharArray())
            if (!Character.isLetter(c) && c != ' ')
                throw new IllegalArgumentException("Artwork name must contain
only letters and spaces.");

        // --- validate price ---
        double price = Double.parseDouble(priceText);
        if (price <= 0) throw new IllegalArgumentException("Price must be a
positive number.");

        // --- find visitor ---
        ArtGalleryVisitor target = null;
        for (ArtGalleryVisitor v : visitorList) {
            if (v.getVisitorId() == visitorId) { target = v; break; }
        }
        if (target == null) throw new IllegalArgumentException("Visitor ID not
found.");

        if (!target.isBought())
            throw new IllegalArgumentException("This visitor has not purchased
any artwork.");
```

```
        boolean nameMatches = target.getArtworkName() != null
            && target.getArtworkName().equalsIgnoreCase(nameText);
        boolean priceMatches = Math.abs(target.getArtworkPrice() - price) <
1e-6;

        if (!nameMatches || !priceMatches)
            throw new IllegalArgumentException("No matching artwork found for
given name and price.");

        // --- domain cancel (handles refund/limits/cleanup) ---
        String msg = target.cancelProduct(visitorId, nameText, reason);

        // --- INLINE row removal from the table model (no helper) ---
        for (int r = model.getRowCount() - 1; r >= 0; r--) {
            Object rid = model.getValueAt(r, 0);
            Object rname = model.getValueAt(r, 1);
            if (rid != null && rname != null
                && rid.toString().equals(String.valueOf(visitorId))
                && rname.toString().equalsIgnoreCase(nameText)) {
                model.removeRow(r);
                break;
            }
        }

        JOptionPane.showMessageDialog(panel,
            msg + "\nRefund: Rs. " + target.getRefundableAmount(),
            "Cancelled", JOptionPane.INFORMATION_MESSAGE);

    } catch (NumberFormatException ex) {
```

```
        JOptionPane.showMessageDialog(panel, "Visitor ID and Price must be  
valid numbers.",  
        "Input Error", JOptionPane.ERROR_MESSAGE);  
    } catch (IllegalArgumentException ex) {  
        JOptionPane.showMessageDialog(panel, ex.getMessage(),  
        "Validation Error", JOptionPane.ERROR_MESSAGE);  
    } catch (Exception ex) {  
        JOptionPane.showMessageDialog(panel, "Unexpected error: " +  
ex.getMessage(),  
        "Error", JOptionPane.ERROR_MESSAGE);  
    }  
}  
}  
});
```

```
btnBill.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        String visitorIdText = txtVisitorId.getText().trim();  
        if (visitorIdText.isEmpty()) {  
            JOptionPane.showMessageDialog(panel, "Enter Visitor ID.", "Error",  
JOptionPane.ERROR_MESSAGE);  
            return;  
        }  
  
        try {  
            // Validate Visitor ID: digits only  
            for (char c : visitorIdText.toCharArray()) {  
                if (!Character.isDigit(c)) {
```

```
        throw new IllegalArgumentException("Visitor ID must contain only
digits.");
    }
}

int visitorId = Integer.parseInt(visitorIdText);

// Find the visitor
ArtGalleryVisitor target = null;
for (ArtGalleryVisitor v : visitorList) {
    if (v.getVisitorId() == visitorId) {
        target = v;
        break;
    }
}
if (target == null) {
    throw new IllegalArgumentException("Visitor ID not found.");
}

// Ensure visitor has purchased
if (!target.isBought) {
    throw new IllegalArgumentException("This visitor has not purchased
any artwork.");
}

// Use single artwork fields (same as cancel)
String artworkName = target.artworkName;
double artworkPrice = target.artworkPrice;
```

```
// Calculate discount & final

double discount = target.calculateDiscount();

double finalAmount = artworkPrice - discount;


// Save invoice

JFileChooser fc = new JFileChooser();

fc.setSelectedFile(new File("Invoice_" + visitorId + ".txt"));

if (fc.showSaveDialog(panel) != JFileChooser.APPROVE_OPTION)

return;


try (PrintWriter out = new PrintWriter(new OutputStreamWriter(
    new FileOutputStream(fc.getSelectedFile()),
    java.nio.charset.StandardCharsets.UTF_8))) {

    out.println("===== AB Art Gallery Invoice =====");
    out.println("Visitor ID : " + target.getVisitorId());
    out.println("Name      : " + target.getFullName());
    out.println("Ticket   : " + target.getTicketType());
    out.println("-----");
    out.println("Artwork   : " + artworkName);
    out.println("Price     : Rs. " + artworkPrice);
    out.println("-----");
    out.println("Discount  : Rs. " + discount);
    out.println("Final Bill : Rs. " + finalAmount);
    out.println("=====");

}
```

```
JOptionPane.showMessageDialog(panel, "Invoice generated  
successfully.", "Success", JOptionPane.INFORMATION_MESSAGE);

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(panel, "Visitor ID must be numeric.",  
"Input Error", JOptionPane.ERROR_MESSAGE);
    } catch (IllegalArgumentException ex) {
        JOptionPane.showMessageDialog(panel, ex.getMessage(), "Validation  
Error", JOptionPane.ERROR_MESSAGE);
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(panel, "Error writing invoice: " +  
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }
}

});
```

```
btnClear.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        txtVisitorId.setText("");
        txtName.setText("");
        txtPrice.setText("");
        txtCancelReason.setText("");
    }
});
```

```
btnDiscount.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String idText = txtVisitorId.getText().trim();
```

```
        if (idText.isEmpty()) {
            JOptionPane.showMessageDialog(panel, "Please enter Visitor ID.",
"Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        try {
            int id = Integer.parseInt(idText);
            boolean found = false;

            for (ArtGalleryVisitor visitor : visitorList) {
                if (visitor.getVisitorId() == id) {
                    found = true;

                    if (!visitor.isBought()) {
                        throw new IllegalArgumentException("This visitor has not
purchased any artwork yet.");
                    }

                    double discount = visitor.calculateDiscount();
                    JOptionPane.showMessageDialog(panel,
                        "Discount: Rs. " + discount,
                        "Discount Info", JOptionPane.INFORMATION_MESSAGE);
                    break;
                }
            }
        }
```

```
        if (!found) {  
            JOptionPane.showMessageDialog(panel, "Visitor ID not found.",  
"Error", JOptionPane.ERROR_MESSAGE);  
        }  
  
    } catch (NumberFormatException ex) {  
        JOptionPane.showMessageDialog(panel, "Visitor ID must be a number.",  
"Input Error", JOptionPane.ERROR_MESSAGE);  
    } catch (IllegalArgumentException ex) {  
        JOptionPane.showMessageDialog(panel, ex.getMessage(), "Validation  
Error", JOptionPane.ERROR_MESSAGE);  
    } catch (Exception ex) {  
        JOptionPane.showMessageDialog(panel, "Unexpected error: " +  
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);  
    }  
}  
});
```

```
btnReward.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        String idText = txtVisitorId.getText().trim();  
  
        if (idText.isEmpty()) {  
            JOptionPane.showMessageDialog(panel, "Please enter Visitor ID.",  
"Error", JOptionPane.ERROR_MESSAGE);  
            return;  
        }  
  
        try {
```

```
int id = Integer.parseInt(idText);

boolean found = false;

for (ArtGalleryVisitor visitor : visitorList) {
    if (visitor.getVisitorId() == id) {
        found = true;

        if (!visitor.isBought()) {
            throw new IllegalArgumentException("This visitor has not
purchased any artwork yet.");
        }

        double reward = visitor.calculateRewardPoint();
        JOptionPane.showMessageDialog(panel,
            "Reward Points: " + reward,
            "Reward Info", JOptionPane.INFORMATION_MESSAGE);
        break;
    }
}

if (!found) {
    JOptionPane.showMessageDialog(panel, "Visitor ID not found.",
"Error", JOptionPane.ERROR_MESSAGE);
}

} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(panel, "Visitor ID must be a number.",
"Input Error", JOptionPane.ERROR_MESSAGE);
}
```

```
        } catch (IllegalArgumentException ex) {

            JOptionPane.showMessageDialog(panel, ex.getMessage(), "Validation
Error", JOptionPane.ERROR_MESSAGE);

        } catch (Exception ex) {

            JOptionPane.showMessageDialog(panel, "Unexpected error: " +
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);

        }

    }

});

int y = 0;

gbc.gridx = 0; gbc.gridy = y; formPanel.add(lblVisitorId, gbc); gbc.gridx = 1;
formPanel.add(txtVisitorId, gbc);

gbc.gridx = 0; gbc.gridy = ++y; formPanel.add(lblName, gbc); gbc.gridx = 1;
formPanel.add(txtName, gbc);

gbc.gridx = 0; gbc.gridy = ++y; formPanel.add(lblPrice, gbc); gbc.gridx = 1;
formPanel.add(txtPrice, gbc);

gbc.gridx = 0; gbc.gridy = ++y; formPanel.add(lblCancelReason, gbc); gbc.gridx =
1; formPanel.add(txtCancelReason, gbc);

gbc.gridx = 0; gbc.gridy = ++y; formPanel.add(btnBuy, gbc); gbc.gridx = 1;
formPanel.add(btnCancel, gbc);

gbc.gridx = 0; gbc.gridy = ++y; formPanel.add(btnBill, gbc); gbc.gridx = 1;
formPanel.add(btnClear, gbc);

gbc.gridx = 0; gbc.gridy = ++y; formPanel.add(btnReward, gbc); gbc.gridx = 1;
formPanel.add(btnDiscount, gbc);

panel.add(formPanel, BorderLayout.NORTH);

panel.add(tableScrollPane, BorderLayout.CENTER);

return panel;

}
```

```
public JButton createNavButton(String text) {  
    JButton button = new JButton(text);  
    button.setBackground(new Color(24, 24, 24));  
    button.setForeground(Color.WHITE);  
    button.setFocusPainted(false);  
    return button;  
}  
  
public static void main(String[] args) {  
    new ArtGalleryGUI();  
}  
}
```